Parameter Incremental Learning Algorithm for Neural Networks

Sheng Wan and Larry E. Banta, *Senior Member, IEEE*

***Abstract*—In this paper, a novel stochastic (or online) training algorithm for neural networks, named parameter incremental learning (PIL) algorithm, is proposed and developed. The main idea of the PIL strategy is that the learning algorithm should not only adapt to the newly presented input–output training pattern by adjusting parameters, but also preserve the prior results. A general PIL algorithm for feedforward neural networks is accordingly presented as the first-order approximate solution to an optimization problem, where the performance index is the combination of proper measures of preservation and adaptation. The PIL algorithms for the multilayer perceptron (MLP) are subsequently derived. Numerical studies show that for all the three benchmark problems used in this paper the PIL algorithm for MLP is measurably superior to the standard online backpropagation (BP) algorithm and the stochastic diagonal Levenberg–Marquardt (SDLM) algorithm in terms of the convergence speed and accuracy. Other appealing features of the PIL algorithm are that it is computationally as simple as the BP algorithm, and as easy to use as the BP algorithm. It, therefore, can be applied, with better performance, to any situations where the standard online BP algorithm is applicable.**

***Index Terms*—Backpropagation (BP), gradient descent, incremental learning, natural gradient descent (NGD), neural networks, online learning.**

I. Introduction

THE problem of learning from input–output data, or so-called *supervised learning*, from the view point of function approximation, can be cast as a problem of using a known function, or so-called *approximator*, with a set of adjustable parameters to approximate an (unknown) *underlying function* through the observed data. To the best knowledge of the authors, all the (parameterized) supervised learning algorithms in the literature are derived from some sort of optimization problems over the adjustable parameter set, where the objective is to optimize a certain cost function (or *error function*) which measures the discrepancy between the observed output and the approximated output with given input. The most frequently used technique in searching solutions is the gradient–descent algorithm, particularly the stochastic (or online) gradient–descent algorithm. The gradient–descent learning algorithm and many of its variants have been extensively investigated and widely practiced, see e.g., [1]–[4] and the references therein. It is worth noting that although many of the variants of the gradient–descent algorithm and other "advanced algorithms" have been proposed, the basic gradient–descent algorithm, though often criticized as slow in convergence, still remains most popular for its simplicity, efficiency, and robustness.

In a gradient–descent learning algorithm, the prediction error of the neural network is reduced after an iteration of applying the learning algorithm over the cost function. No specific measure is explicitly taken to preserve the "already learned" property of the approximator during the learning iteration. On the other hand, in view of the learning approaches of human beings, it seems natural to build successive learning upon prior results. This is essentially a feature of *incremental learning*. It basically means that after the approximator makes an adaptation to the new input–output pattern, two goals are simultaneously achieved: First, the adaptation, i.e., the prediction error of the neural network is reduced (if not completely eliminated) for this new pattern; second, the preservation, i.e., the perturbation of the approximator caused by the adaptation is kept as small as possible, thus the *a priori* result is best preserved. It is noted that the term "incremental learning" has been widely used in the literature, mostly to refer to those techniques where some sort of structural adaptation is involved to achieve the goal of incremental learning (e.g., see [5], [6], and the references therein). For example, in the well-known resource allocation networks proposed in [7] for the radial basis network, the spirit of incremental learning is achieved mainly by recruiting a new neuron, or allocating a new *resource*, whenever necessary. The incremental learning may also be addressed in the context of training data manipulation. For example, in [8], an incremental learning strategy is implemented through the selection of most informative training samples. To the best knowledge of the authors, no work has been done that addresses the issue of incremental learning only in the context of parameter update laws. On the other hand, it appears that most of the known parameter update laws of learning algorithms such as gradient descent only address the first goal of incremental learning, i.e., adaptation to the training data, without explicitly addressing the issue of preservation of *a priori* results.

In this paper, a supervised learning approach is proposed which simultaneously addresses the two goals of incremental learning in a unified framework and aims at improving the efficiency of the learning algorithm. We note that unlike other incremental learning methods, this paper concentrates on the situation where only parameter adaptation is considered. Thus, the proposed method is called the *parameter incremental learning* (PIL) strategy.

Manuscript received May 18, 2005; revised January 5, 2006.

S. Wan was with the Mechanical and Aerospace Engineering Department, West Virginia University, Morgantown, WV 26505 USA. He is now with GE Transportation, Erie, PA 16531 USA (e-mail: Sheng_Wan@yahoo.com).

L. E. Banta is with the Mechanical and Aerospace Engineering Department, West Virginia University, Morgantown, WV 26505 USA (e-mail: Larry.Banta@mail.wvu.edu).

Color versions of Figs. 1–8 are available online at http://ieeexplore.ieee.org.

Digital Object Identifier 10.1109/TNN.2006.880581

1045-9227/$20.00 © 2006 IEEE

The rest of this paper is organized as follows. Section II formulates the PIL problem and the first-order approximate solution is subsequently presented. In Section III, a method of selecting the free design parameters of the PIL algorithm is proposed. In Section IV, the PIL algorithm for the MLP is derived from the general PIL algorithm. Section V is devoted to numerical simulations, demonstrating the advantages of the new algorithm over the BP algorithm and the stochastic diagonal Levenberg–Marquardt (SDLM) algorithm. Finally, the main features and advantages of the new algorithm are summarized in Section VI.

II. Parameter Incremental Learning Problem

The following notations will be used in this paper. $\mathbf{\Psi}(\cdot,\boldsymbol{\theta}) : \Re^{\ell} \rightarrow \Re^{m}$ is a smooth function representing the map of a neural network, or an *approximator*, with adjustable parameter $\boldsymbol{\theta} \in \Re^{M}$; $g_i(\cdot,\boldsymbol{\theta}_i) : \Re^{n_i} \rightarrow \Re$ is a smooth function denoting the ith neuron of a neural network with $\boldsymbol{\theta}_i$ being its adjustable parameter vector, $i = 1, 2, \ldots, L$, where L is the total number of neurons in the neural network; $m(\cdot,\cdot)$ denotes a distance metric between two functions; $\|\mathbf{x}\| = (\mathbf{x}^{\mathbf{T}}\mathbf{x})^{1/2}$ denotes the Euclidean norm of a vector, while $\|\mathbf{x}\|_{\mathbf{M}} = (\mathbf{x}^{\mathbf{T}}\mathbf{M}\mathbf{x})^{1/2}, \mathbf{M} > 0$ (i.e., positive–definite) denotes the weighted Euclidean norm. The approximate equality notation $\cong$ is used throughout this paper to denote that two quantities are approximately equal in the sense of first-order approximation. $\mathbf{I}$ stands for the identity matrix with appropriate dimension. A quantity $\mathbf{F}(\mathbf{x})$, which depends on another quantity $\mathbf{X}$, will sometimes be abbreviated as $\mathbf{F}$, where no confusion is likely.

PIL is basically a tradeoff between the adaptation to the new input–output pattern and the preservation of the *a priori* result of the neural network when the learning algorithm responds to the new input–output patterns. In this regard, two performance indices need to be simultaneously addressed: The performance of adaptation and the performance of preservation. For a given input–output pair of training data $(\mathbf{x}, \mathbf{y})$, the weighted squared error between the target value and the predicted output by the neural network is chosen to measure the performance of adaptation, expressed as

$$J^{\mathrm{adpt}} = \frac{1}{2}\mathbf{\Delta y}_{\mathbf{new}}^{\mathbf{T}}\Xi\mathbf{\Delta y}_{\mathbf{new}} \tag{1}$$

where the matrix Ξ is a positive–definite weight matrix, and

$$\Delta\mathbf{y}_{\mathbf{new}} = \mathbf{\Psi}(\mathbf{x}, \theta + \delta\boldsymbol{\theta}) - \mathbf{y} \tag{2}$$

is the *postprediction error*, i.e., the prediction error of the neural network after the parameter is updated from $\boldsymbol{\theta}$ to $\theta + \delta\boldsymbol{\theta}$. On the other hand, the performance index measuring the neural network's deformation caused by the parameter adaptation is chosen to be the commonly used squared L^2-norm induced metric of the distance between functions, i.e.,

$$J^{\mathrm{pres}} = \frac{1}{2}m_{L^2}^2(\mathbf{\Psi}(\mathbf{x}, \boldsymbol{\theta} + \delta\boldsymbol{\theta}), \mathbf{\Psi}(\mathbf{x}, \boldsymbol{\theta})) \tag{3}$$

where

$$m_{L^2}(\mathbf{\Psi}(\mathbf{x}, \boldsymbol{\theta} + \delta\boldsymbol{\theta}), \mathbf{\Psi}(\mathbf{x}, \boldsymbol{\theta})) = \left(\int_{\mathbf{x}\in\mathcal{D}} \|\mathbf{\Psi}(\mathbf{x}, \boldsymbol{\theta} + \delta\boldsymbol{\theta}) - \mathbf{\Psi}(\mathbf{x}, \boldsymbol{\theta})\|^2 d\mathbf{x}\right)^{1/2} \tag{4}$$

and $\mathcal{D}$ is a properly defined region of the input to the network (to be discussed in Section IV). Ideally, we want both J^{adpt} and J^{pres} be as small as possible after the parameter update. Thus, the cost function that captures the essence of the PIL, which is basically a tradeoff between the two (somewhat contradictive) performance indices, is obtained by combining the two performance indices (1) and (3) as the following single cost function:

$$J = J^{\mathrm{adpt}} + \lambda J^{\mathrm{pres}} \tag{5}$$

where $\lambda > 0$. The general PIL problem (at the global level) can thus be stated as: Given a pair of input–output training vectors $(\mathbf{x}, \mathbf{y})$, find the parameter increment $\delta\boldsymbol{\theta}$ to the modifiable parameter $\boldsymbol{\theta}$ of the neural network $\mathbf{\Psi}$ such that the cost function J given by (5) is minimized.

Note that our purpose here is to obtain a learning algorithm by deriving an (approximate) analytical solution to the optimization problem. However, it is extremely difficult to find such a solution (or an approximate solution) to this problem for general neural networks with nonlinear neurons. To overcome the difficulty, a suboptimal strategy is proposed. The key idea of the strategy is to define an index which measures the neural network preservation performance at the neuron level, instead of at the "global level" as is defined by (3). This implies that the performance index is measured against the deformation of the individual neurons instead of against the entire neural network as a whole. A suitable performance index, which will simplify the problem while maintaining the essence of the preservation feature of the PIL, is

$$\tilde{J}^{\mathrm{pres}} = \frac{1}{2}\sum_{i=1}^{L}\lambda_i m_{L^2}^2(g_i(\mathbf{u}_i, \boldsymbol{\theta}_i + \delta\boldsymbol{\theta}_i), g_i(\mathbf{u}_i, \boldsymbol{\theta}_i)) \tag{6}$$

where the distance metric

$$m_{L^2}(g_i(\mathbf{u}_i, \boldsymbol{\theta}_i + \delta\boldsymbol{\theta}_i), g_i(\mathbf{u}_i, \boldsymbol{\theta}_i)) = \left(\int_{\mathbf{u}_i\in\mathcal{D}_\rangle} (g_i(\mathbf{u}_i, \boldsymbol{\theta}_i + \delta\boldsymbol{\theta}_i) - g_i(\mathbf{u}_i, \boldsymbol{\theta}_i))^2 d\mathbf{u}_i\right)^{\frac{1}{2}}, \quad i = 1, 2, \ldots, L \tag{7}$$

is a measure of the "deformation" of a single neuron caused by its own parameter perturbation. $\lambda_i (i = 1, \ldots, L)$ are positive numbers (to be designed). $\mathbf{u}_i$ is the input to the ith neuron $\mathbf{g}_i$. It is assumed that $(g_i(\mathbf{u}_i, \boldsymbol{\theta}_i + \delta\boldsymbol{\theta}_i) - g_i(\mathbf{u}_i, \boldsymbol{\theta}_i))^2$ is integrable over a properly defined region $\mathcal{D}_\rangle$ for each $i = 1, \ldots, L$. It will be seen in Section IV that with the proposed performance index

given by (6) and (7), the PIL algorithm for MLP neural network is solvable (with the aid of some mathematical tricks).

Thus, the following general PIL problem (at the neuron level) is posed.

General PIL Problem: Given a pair of input–output training data vectors $(\mathbf{x}, \mathbf{y})$, find the parameter increment $\delta\boldsymbol{\theta}$ to the modifiable parameter $\boldsymbol{\theta}$ of the neural network $\boldsymbol{\Psi}(\cdot, \boldsymbol{\theta})$ such that the following cost function is minimized:

$$\tilde{J} = \frac{1}{2}\boldsymbol{\Delta}\mathbf{y}_{\mathbf{new}}^{\mathbf{T}}\Xi\boldsymbol{\Delta}\mathbf{y}_{\mathbf{new}} + \frac{1}{2}\sum_{i=1}^{L}\lambda_i \int_{\mathbf{u}_i \in \mathcal{D}_\rangle} (g_i(\mathbf{u}_i, \boldsymbol{\theta}_i + \delta\boldsymbol{\theta}_i) - g_i(\mathbf{u}_i, \boldsymbol{\theta}_i))^2 d\mathbf{u}_i. \quad (8)$$

This is generally a nonlinear optimization problem where again an analytical solution is hard to obtain. However, this complex nonlinear optimization problem can be easily "solved" when using the first-order approximation of the neural network with respect to the parameter perturbation, as is given by Theorem 1.

Theorem 1 (General PIL Algorithm): The first-order approximate solution to the general PIL problem is given by

$$\delta\boldsymbol{\theta}_i^{\mathbf{PIL}} \triangleq -\frac{1}{\lambda_i}\boldsymbol{\Theta}_i^{-1}(\boldsymbol{\theta}_i)\frac{\partial\boldsymbol{\Psi}(\mathbf{x}, \boldsymbol{\theta})}{\partial\boldsymbol{\theta}_i}\Omega\Delta\mathbf{y}, \qquad i = 1, \ldots, L \quad (9)$$

where

$$\Delta\mathbf{y} \triangleq \boldsymbol{\Psi}(\mathbf{x}, \boldsymbol{\theta}) - \mathbf{y} \quad (10)$$

is the *a priori–prediction error*, i.e., the prediction error of the neural network prior to parameter updating, and

$$\boldsymbol{\Theta}_i(\boldsymbol{\theta}_i) \triangleq \int_{\mathbf{u}_i \in \mathcal{D}_\rangle} \frac{\partial g_i(\mathbf{u}_i, \boldsymbol{\theta}_i)}{\partial\boldsymbol{\theta}_i}\left(\frac{\partial g_i(\mathbf{u}_i, \boldsymbol{\theta}_i)}{\partial\boldsymbol{\theta}_i}\right)^{\mathbf{T}} d\mathbf{u}_i, \qquad i = 1, \ldots, L \quad (11)$$

$$\Omega \triangleq \left(\Xi^{-1} + \left(\frac{\partial\boldsymbol{\Psi}(\mathbf{x}, \boldsymbol{\theta})}{\partial\boldsymbol{\theta}}\right)^{\mathbf{T}}\boldsymbol{\Theta}_\lambda^{-1}(\boldsymbol{\theta})\frac{\partial\boldsymbol{\Psi}(\mathbf{x}, \boldsymbol{\theta})}{\partial\boldsymbol{\theta}}\right)^{-1} \quad (12)$$

assuming that each $\boldsymbol{\Theta}_i(\boldsymbol{\theta}_i)$ is invertible, and

$$\boldsymbol{\Theta}_\lambda(\boldsymbol{\theta}) \triangleq \mathrm{diag}[\lambda_1\boldsymbol{\Theta}_1 \quad \lambda_2\boldsymbol{\Theta}_2 \quad \cdots \quad \lambda_L\boldsymbol{\Theta}_L]. \quad (13)$$

Proof: Note that by using the first-order approximation of the neural network with respect to the parameter perturbation, we have

$$\boldsymbol{\Psi}(\mathbf{x}, \boldsymbol{\theta} + \delta\boldsymbol{\theta}) \cong \boldsymbol{\Psi}(\mathbf{x}, \boldsymbol{\theta}) + \left(\frac{\partial\boldsymbol{\Psi}(\mathbf{x}, \boldsymbol{\theta})}{\partial\boldsymbol{\theta}}\right)^{\mathbf{T}}\delta\boldsymbol{\theta}. \quad (14)$$

Then

$$\begin{aligned}\Delta\mathbf{y}_{\mathbf{new}} &= \boldsymbol{\Psi}(\mathbf{x}, \boldsymbol{\theta} + \delta\boldsymbol{\theta}) - \mathbf{y} \\ &= (\boldsymbol{\Psi}(\mathbf{x}, \boldsymbol{\theta} + \delta\boldsymbol{\theta}) - \boldsymbol{\Psi}(\mathbf{x}, \boldsymbol{\theta})) + (\boldsymbol{\Psi}(\mathbf{x}, \boldsymbol{\theta}) - \mathbf{y}) \\ &\cong \left(\frac{\partial\boldsymbol{\Psi}(\mathbf{x}, \boldsymbol{\theta})}{\partial\boldsymbol{\theta}}\right)^{\mathbf{T}}\delta\boldsymbol{\theta} + \Delta\mathbf{y}. \end{aligned} \quad (15)$$

On the other hand, by using the first-order approximation of the neuron with respect to the parameter perturbation, we have

$$g_i(\mathbf{u}_i, \boldsymbol{\theta}_i + \delta\boldsymbol{\theta}_i) \cong g_i(\mathbf{u}_i, \boldsymbol{\theta}_i) + \left(\frac{\partial g_i(\mathbf{u}_i, \boldsymbol{\theta}_i)}{\partial\boldsymbol{\theta}_i}\right)^{\mathbf{T}}\delta\boldsymbol{\theta}_i, \qquad i = 1, \ldots, L. \quad (16)$$

Thus, the equation (7) is approximated as

$$\begin{aligned} &m_{L^2}^2(g_i(\mathbf{u}_i, \boldsymbol{\theta}_i + \delta\boldsymbol{\theta}_i), g_i(\mathbf{u}_i, \boldsymbol{\theta}_i)) \\ &\cong \int_{\mathbf{u}_i \in \mathcal{D}_\rangle} \left(\left(\frac{\partial g_i(\mathbf{u}_i, \boldsymbol{\theta}_i)}{\partial\boldsymbol{\theta}_i}\right)^{\mathbf{T}}\delta\boldsymbol{\theta}_i\right)^{\mathbf{T}} \\ &\quad \times \left(\left(\frac{\partial g_i(\mathbf{u}_i, \boldsymbol{\theta}_i)}{\partial\boldsymbol{\theta}_i}\right)^{\mathbf{T}}\delta\boldsymbol{\theta}_i\right) d\mathbf{u}_i \\ &= \delta\boldsymbol{\theta}_i^{\mathbf{T}}\left(\int_{\mathbf{u}_i \in \mathcal{D}_\rangle} \frac{\partial g_i(\mathbf{u}_i, \boldsymbol{\theta}_i)}{\partial\boldsymbol{\theta}_i}\left(\frac{\partial g_i(\mathbf{u}_i, \boldsymbol{\theta}_i)}{\partial\boldsymbol{\theta}_i}\right)^{\mathbf{T}} d\mathbf{u}_i\right)\delta\boldsymbol{\theta}_i \\ &= \delta\boldsymbol{\theta}_i^{\mathbf{T}}\boldsymbol{\Theta}_i(\boldsymbol{\theta}_i)\delta\boldsymbol{\theta}_i, \qquad i = 1, \ldots, L. \end{aligned} \quad (17)$$

Also note that the adjustable parameter vector $\boldsymbol{\theta}$ to the entire neural network can be written as the concatenation of all the parameter vectors of each of the neurons in the neural network, i.e.,

$$\boldsymbol{\theta} = \begin{bmatrix}\boldsymbol{\theta}_1^{\mathbf{T}} & \boldsymbol{\theta}_2^{\mathbf{T}} & \cdots & \boldsymbol{\theta}_L^{\mathbf{T}}\end{bmatrix}^{\mathbf{T}}. \quad (18)$$

Then, the index of preservation performance (6) can be approximated as

$$\tilde{J}^{\mathrm{pres}} \cong \frac{1}{2}\sum_{i=1}^{L}\lambda_i\delta\boldsymbol{\theta}_i^{\mathbf{T}}\boldsymbol{\Theta}_i(\boldsymbol{\theta}_i)\delta\boldsymbol{\theta}_i = \frac{1}{2}\delta\boldsymbol{\theta}^{\mathbf{T}}\boldsymbol{\Theta}_\lambda(\boldsymbol{\theta})\delta\boldsymbol{\theta} \quad (19)$$

Therefore, the cost function (8) can be approximated as

$$\begin{aligned}\tilde{J} \cong \frac{1}{2}&\left(\left(\frac{\partial\boldsymbol{\Psi}(\mathbf{x}, \boldsymbol{\theta})}{\partial\boldsymbol{\theta}}\right)^{\mathbf{T}}\delta\boldsymbol{\theta} + \Delta\mathbf{y}\right)^{\mathbf{T}} \\ &\Xi\left(\left(\frac{\partial\boldsymbol{\Psi}(\mathbf{x}, \boldsymbol{\theta})}{\partial\boldsymbol{\theta}}\right)^{\mathbf{T}}\delta\boldsymbol{\theta} + \Delta\mathbf{y}\right) + \frac{1}{2}\delta\boldsymbol{\theta}^{\mathbf{T}}\boldsymbol{\Theta}_\lambda(\boldsymbol{\theta})\delta\boldsymbol{\theta}. \end{aligned} \quad (20)$$

The optimal solution to the *quadratic optimization problem* with the cost function given by (20) can be obtained from the following optimality condition:

$$\frac{\partial\tilde{J}}{\partial\delta\boldsymbol{\theta}} \cong \frac{\partial\boldsymbol{\Psi}(\mathbf{x}, \boldsymbol{\theta})}{\partial\boldsymbol{\theta}}\Xi\left(\left(\frac{\partial\boldsymbol{\Psi}(\mathbf{x}, \boldsymbol{\theta})}{\partial\boldsymbol{\theta}}\right)^{\mathbf{T}}\delta\boldsymbol{\theta} + \Delta\mathbf{y}\right) + \boldsymbol{\Theta}_\lambda(\boldsymbol{\theta})\delta\boldsymbol{\theta} \triangleq 0. \quad (21)$$

The solution to (21) is given by

$$\begin{aligned}\delta\boldsymbol{\theta}^{\mathbf{PIL}} &= -\left(\frac{\partial\boldsymbol{\Psi}(\mathbf{x}, \boldsymbol{\theta})}{\partial\boldsymbol{\theta}}\Xi\left(\frac{\partial\boldsymbol{\Psi}(\mathbf{x}, \boldsymbol{\theta})}{\partial\boldsymbol{\theta}}\right)^{\mathbf{T}} + \boldsymbol{\Theta}_\lambda(\boldsymbol{\theta})\right)^{-1} \\ &\quad \times \frac{\partial\boldsymbol{\Psi}(\mathbf{x}, \boldsymbol{\theta})}{\partial\boldsymbol{\theta}}\Xi\Delta\mathbf{y} \\ &= -\boldsymbol{\Theta}_\lambda^{-1}(\boldsymbol{\theta})\frac{\partial\boldsymbol{\Psi}(\mathbf{x}, \boldsymbol{\theta})}{\partial\boldsymbol{\theta}}\left(\Xi^{-1} + \left(\frac{\partial\boldsymbol{\Psi}(\mathbf{x}, \boldsymbol{\theta})}{\partial\boldsymbol{\theta}}\right)^{\mathbf{T}}\right. \end{aligned}$$

$$\times\boldsymbol{\Theta}_{\lambda}^{-1}(\boldsymbol{\theta})\frac{\partial\boldsymbol{\Psi}(\mathbf{x},\boldsymbol{\theta})}{\partial\boldsymbol{\theta}}\Big)^{-1}\Delta\mathbf{y}$$
$$=-\boldsymbol{\Theta}_{\lambda}^{-1}(\boldsymbol{\theta})\frac{\partial\boldsymbol{\Psi}(\mathbf{x},\boldsymbol{\theta})}{\partial\boldsymbol{\theta}}\Omega\Delta\mathbf{y} \tag{22}$$

and this is equivalent to (9). This completes the proof.

To gain further insights for the PIL algorithm, it is of interest to take a look at the problem from an alternative perspective. In the following, the PIL problem will be posed from a different perspective yet is able to capture the same essence of incremental learning as previously discussed. The main point here is to pose the problem as if only a single neuron in the network were to be adapted when a training pattern is presented, and the performance index which bears the essence of incremental learning is posed so as to minimize the deformation of this single neuron, subject to a certain amount of error reduction (i.e., adaptation to the training pattern) specified by an equality constraint. Quite interestingly, it will be shown in what follows that, though started from a different perspective, the resulting algorithm is essentially the same as the one obtained in Theorem 1.

To simplify the presentation, it is assumed that the neural network has a single output. The alternative version of the PIL problem at the neuron level is posed as follows.

Alternative Version of the General PIL Problem: Given a new pair of input–output training data $(\mathbf{x},\mathbf{y})$, find the increment of the parameter vector $\boldsymbol{\theta}_i$ of the ith neuron in the neural network $\delta\boldsymbol{\theta}_i$ that minimizes the cost function, or the deformation of the neuron caused by the variation of the parameter vector

$$J_i(\boldsymbol{\theta}_i,\delta\boldsymbol{\theta}_i)=\frac{1}{2}\int\limits_{\mathbf{u}_i\in\mathcal{D}_\rangle}(g_i(\mathbf{u}_i,\boldsymbol{\theta}_i+\delta\boldsymbol{\theta}_i)-g_i(\mathbf{u}_i,\boldsymbol{\theta}_i))^2d\mathbf{u}_i \tag{23}$$

subject to the equality constraint

$$\Psi(\mathbf{x},\boldsymbol{\theta}_i+\delta\boldsymbol{\theta}_i)-\Psi(\mathbf{x},\boldsymbol{\theta}_i)=-\kappa_i\Delta y \tag{24}$$

where $1\gg\kappa_i>0$ is a user selected parameter which specifies the amount of error reduction in response to parameter adaptation.

Note that with a small positive number κ_i, the constraint (24) guarantees the prediction error to be reduced after the parameter is updated from $\boldsymbol{\theta}_i$ to $\boldsymbol{\theta}_i+\delta\boldsymbol{\theta}_i$, since

$$\begin{aligned}\Delta y_{\mathbf{new}}^{(i)}&\triangleq\Psi(\mathbf{x},\boldsymbol{\theta}_i+\delta\boldsymbol{\theta}_i)-y\\&=\Psi(\mathbf{x},\boldsymbol{\theta}_i+\delta\boldsymbol{\theta}_i)-\Psi(\mathbf{x},\boldsymbol{\theta}_i)+\Psi(\mathbf{x},\boldsymbol{\theta}_i)-y\\&=-\kappa_i\Delta y+\Delta y=(1-\kappa_i)\Delta y\end{aligned} \tag{25}$$

which implies

$$\|\Delta y_{\mathbf{new}}^{(i)}\|=\|(1-\kappa_i)\Delta y\|=\|(1-\kappa_i)\|\|\Delta y\|<\|\Delta y\|. \tag{26}$$

The aforementioned inequality holds since the inequality $1\gg\kappa_i>0$ (by assumption) implies $\|(1-\kappa_i)\|<1$.

The first-order approximate solution to the previously posed problem can be solved as stated in Theorem 2.

Theorem 2: The first-order approximate solution of the parameter increment $\delta\boldsymbol{\theta}_i$ that optimizes the performance index (23) subject to the equality constraint (24) is given by

$$\delta\hat{\theta}_i^{\mathbf{PIL}}\triangleq-\frac{\kappa_i}{\left(\frac{\partial\Psi(\mathbf{x},\boldsymbol{\theta}_i)}{\partial\boldsymbol{\theta}_i}\right)^{\mathbf{T}}\boldsymbol{\Theta}_i^{-1}\frac{\partial\Psi(\mathbf{x},\boldsymbol{\theta}_i)}{\partial\boldsymbol{\theta}_i}}\boldsymbol{\Theta}_i^{-1}\frac{\partial\Psi(\mathbf{x},\boldsymbol{\theta}_i)}{\partial\boldsymbol{\theta}_i}\Delta y. \tag{27}$$

Proof: From (17), the performance index (23) can be written as

$$J_i(\boldsymbol{\theta}_i,\delta\boldsymbol{\theta}_i)\cong\frac{1}{2}\delta\boldsymbol{\theta}_i^{\mathbf{T}}\boldsymbol{\Theta}_i\delta\boldsymbol{\theta}_i \tag{28}$$

and by using the first-order approximation similar to (14) (with $\delta\boldsymbol{\theta}$ being replaced by $\delta\boldsymbol{\theta}_i$), the equality constraint (24) becomes

$$\left(\frac{\partial\Psi(\mathbf{x},\boldsymbol{\theta}_i)}{\partial\boldsymbol{\theta}_i}\right)^{\mathbf{T}}\delta\boldsymbol{\theta}_i\cong-\kappa_i\Delta y. \tag{29}$$

Let

$$\tilde{J}(\boldsymbol{\theta}_i,\delta\boldsymbol{\theta}_i)\triangleq\frac{1}{2}\delta\boldsymbol{\theta}_i^{\mathbf{T}}\boldsymbol{\Theta}_i\delta\boldsymbol{\theta}_i+\lambda\left(\left(\frac{\partial\Psi(\mathbf{x},\boldsymbol{\theta}_i)}{\partial\boldsymbol{\theta}_i}\right)^{\mathbf{T}}\delta\boldsymbol{\theta}_i+\kappa_i\Delta y\right) \tag{30}$$

where λ is the Lagrange multiplier. The optimal $\delta\boldsymbol{\theta}_i$ is then obtained by the following optimization condition:

$$\frac{\partial\tilde{J}(\boldsymbol{\theta}_i,\delta\boldsymbol{\theta}_i)}{\partial\delta\boldsymbol{\theta}_i}=\boldsymbol{\Theta}_i\delta\boldsymbol{\theta}_i+\lambda\frac{\partial\Psi(\mathbf{x},\boldsymbol{\theta}_i)}{\partial\boldsymbol{\theta}_i}\triangleq 0. \tag{31}$$

This leads to

$$\delta\boldsymbol{\theta}_i=-\lambda\boldsymbol{\Theta}_i^{-1}\frac{\partial\Psi(\mathbf{x},\boldsymbol{\theta}_i)}{\partial\boldsymbol{\theta}_i}. \tag{32}$$

Substituting (32) into (29) yields

$$\lambda=\frac{\kappa_i\Delta y}{\left(\frac{\partial\Psi(\mathbf{x},\boldsymbol{\theta}_i)}{\partial\boldsymbol{\theta}_i}\right)^{\mathbf{T}}\boldsymbol{\Theta}_i^{-1}\frac{\partial\Psi(\mathbf{x},\boldsymbol{\theta}_i)}{\partial\boldsymbol{\theta}_i}}. \tag{33}$$

Substituting (33) back into (32) yields the optimal solution of $\delta\boldsymbol{\theta}_i$ given by (27). This completes the proof of Theorem 2.

By comparing (27) with (9) it can be seen that they are essentially the same up to a selection of a free parameter, since both κ_i and λ_i are free parameters, and the Ω in (9) is a scalar for single output neural networks.

III. Selection of Parameters for the General PIL Algorithm

One item remains to be solved in using the general PIL algorithm: The selection of the algorithm parameters $\lambda_i, i=1,2,\ldots,L$, and Ξ. This is clearly a nontrivial task. Unfortunately, there is no readily available theoretical guideline for selecting a set of parameters that could lead to satisfactory learning performance. From the practical point of view, however, a promising and convenient way of dealing with this issue

is to compare the relevant properties of the PIL algorithm with that of the conventional stochastic gradient–descent algorithm since the latter works efficiently in many applications. In what follows, we will make a selection of the parameters of the PIL algorithm by *normalizing* neuron-by-neuron the error-reduction property of the PIL algorithm to that of the standard gradient–descent learning algorithm.

The well-known *standard stochastic (or online) gradient–descent* learning algorithm is given by the following parameter update law:

$$\delta\boldsymbol{\theta}^{\mathbf{GRD}} \triangleq -\mu\frac{\partial E}{\partial\boldsymbol{\theta}} \tag{34}$$

where

$$E(\boldsymbol{\theta}) = \frac{1}{2}\boldsymbol{\Delta}\mathbf{y}^{\mathbf{T}}\boldsymbol{\Delta}\mathbf{y} \tag{35}$$

is the cost function for current training data $(\mathbf{x},\mathbf{y})$, and $\mu > 0$ is the *learning rate*. Note that the spirit of the online gradient learning algorithm is that after the parameter update by (34), the cost function (35) will be (approximately) reduced, i.e.,

$$\begin{aligned} E(\boldsymbol{\theta}+\delta\boldsymbol{\theta}^{\mathbf{GRD}}) - E(\boldsymbol{\theta}) &\cong \left(\delta\boldsymbol{\theta}^{\mathbf{GRD}}\right)^{\mathbf{T}}\frac{\partial E(\boldsymbol{\theta})}{\partial\boldsymbol{\theta}} \\ &= -\mu\|\frac{\partial E(\boldsymbol{\theta})}{\partial\boldsymbol{\theta}}\|^2 \leqslant 0. \end{aligned} \tag{36}$$

To simplify the notation, let us denote the partial differentiation of $g_i(\mathbf{u}_i,\boldsymbol{\theta}_i)$, $\boldsymbol{\Psi}(\mathbf{x},\boldsymbol{\theta})$ with respect to the parameter vector $\boldsymbol{\theta}_i$, the neuron g_i, respectively, as

$$\mathbf{p}_i(\mathbf{u}_i,\boldsymbol{\theta}_i) \triangleq \frac{\partial g_i(\mathbf{u}_i,\boldsymbol{\theta}_i)}{\partial\boldsymbol{\theta}_i} \tag{37}$$

$$\mathbf{q}_i(\mathbf{x},\boldsymbol{\theta}) \triangleq \frac{\partial\boldsymbol{\Psi}(\mathbf{x},\boldsymbol{\theta})}{\partial g_i}. \tag{38}$$

Then, we have Theorem 3.

Theorem 3: If the parameters for the PIL algorithm are chosen as

$$\Xi = \left(\varepsilon^{-1}\mathbf{I} - \left(\frac{\partial\boldsymbol{\Psi}(\mathbf{x},\boldsymbol{\theta})}{\partial\boldsymbol{\theta}}\right)^{\mathbf{T}}\boldsymbol{\Theta}_\lambda^{-1}(\boldsymbol{\theta})\frac{\partial\boldsymbol{\Psi}(\mathbf{x},\boldsymbol{\theta})}{\partial\boldsymbol{\theta}}\right)^{-1} \tag{39}$$

$$\frac{\varepsilon}{\lambda_i} = \mu\frac{\mathbf{p}_i^{\mathbf{T}}\mathbf{p}_i}{\mathbf{p}_i^{\mathbf{T}}\boldsymbol{\Theta}_i^{-1}\mathbf{p}_i}, \qquad i = 1,2,\ldots,L \tag{40}$$

where ε is a sufficiently small positive number such that the Ξ given by (39) is guaranteed to be a positive-definite matrix, then

$$E(\boldsymbol{\theta}+\delta\boldsymbol{\theta}^{\mathbf{GRD}}) \cong E(\boldsymbol{\theta}+\delta\boldsymbol{\theta}^{\mathbf{PIL}}). \tag{41}$$

Furthermore, the PIL algorithm is given by

$$\delta\boldsymbol{\theta}_i^{\mathbf{PIL}} = -\mu\frac{\mathbf{p}_i^{\mathbf{T}}\mathbf{p}_i}{\mathbf{p}_i^{\mathbf{T}}\boldsymbol{\Theta}_i^{-1}\mathbf{p}_i}\boldsymbol{\Theta}_i^{-1}\mathbf{p}_i\left(\mathbf{q}_i^{\mathbf{T}}\boldsymbol{\Delta}\mathbf{y}\right), \qquad i = 1,2,\ldots,L. \tag{42}$$

Proof: Note that

$$\frac{\partial\boldsymbol{\Psi}(\mathbf{x},\boldsymbol{\theta})}{\partial\boldsymbol{\theta}_i} = \frac{\partial g_i(\mathbf{x},\boldsymbol{\theta}_i)}{\partial\boldsymbol{\theta}_i}\left(\frac{\partial\boldsymbol{\Psi}(\mathbf{x},\boldsymbol{\theta})}{\partial g_i}\right)^{\mathbf{T}} = \mathbf{p}_i\mathbf{q}_i^{\mathbf{T}} \tag{43}$$

and

$$\begin{aligned} \frac{\partial E}{\partial\boldsymbol{\theta}} &= \frac{\partial\boldsymbol{\Psi}(\mathbf{x},\boldsymbol{\theta})}{\partial\boldsymbol{\theta}}\boldsymbol{\Delta}\mathbf{y} \\ &= \left[\left(\frac{\partial\boldsymbol{\Psi}(\mathbf{x},\boldsymbol{\theta})}{\partial\boldsymbol{\theta}_1}\right)^{\mathbf{T}},\ldots,\left(\frac{\partial\boldsymbol{\Psi}(\mathbf{x},\boldsymbol{\theta})}{\partial\boldsymbol{\theta}_L}\right)^{\mathbf{T}}\right]^{\mathbf{T}}\boldsymbol{\Delta}\mathbf{y} \\ &= \left[\mathbf{q_1}\mathbf{p_1^T},\ldots,\mathbf{q}_L\mathbf{p}_L^{\mathbf{T}}\right]^{\mathbf{T}}\boldsymbol{\Delta}\mathbf{y}. \end{aligned} \tag{44}$$

Then

$$\begin{aligned} E(\boldsymbol{\theta}+\delta\boldsymbol{\theta}^{\mathbf{GRD}}) &\cong E(\boldsymbol{\theta}) + \left(\frac{\partial E}{\partial\boldsymbol{\theta}}\right)^{\mathbf{T}}\delta\boldsymbol{\theta}^{\mathbf{GRD}} \\ &= E(\boldsymbol{\theta}) - \mu\left(\frac{\partial E}{\partial\boldsymbol{\theta}}\right)^{\mathbf{T}}\frac{\partial E}{\partial\boldsymbol{\theta}} \\ &= E(\boldsymbol{\theta}) - \mu\boldsymbol{\Delta}\mathbf{y}^{\mathbf{T}}\left(\sum_{i=1}^{L}\left(\mathbf{p}_i^{\mathbf{T}}\mathbf{p}_i\right)\mathbf{q}_i\mathbf{q}_i^{\mathbf{T}}\right)\boldsymbol{\Delta}\mathbf{y}. \end{aligned} \tag{45}$$

On the other hand, note that with the Ξ given by (39), we have

$$\Omega = \varepsilon\mathbf{I}. \tag{46}$$

Then

$$\begin{aligned} &E(\boldsymbol{\theta}+\delta\boldsymbol{\theta}^{\mathbf{PIL}}) \\ &\cong E(\boldsymbol{\theta}) + \left(\frac{\partial E}{\partial\boldsymbol{\theta}}\right)^{\mathbf{T}}\delta\boldsymbol{\theta}^{\mathbf{PIL}} \\ &= E(\boldsymbol{\theta}) - \boldsymbol{\Delta}\mathbf{y}^{\mathbf{T}}\left(\frac{\partial\boldsymbol{\Psi}(\mathbf{x},\boldsymbol{\theta})}{\partial\boldsymbol{\theta}}\right)^{\mathbf{T}}\boldsymbol{\Theta}_\lambda^{-1}(\boldsymbol{\theta})\frac{\partial\boldsymbol{\Psi}(\mathbf{x},\boldsymbol{\theta})}{\partial\boldsymbol{\theta}}\Omega\Delta\mathbf{y} \\ &= E(\boldsymbol{\theta}) - \varepsilon\boldsymbol{\Delta}\mathbf{y}^{\mathbf{T}}\begin{bmatrix}\frac{\partial\boldsymbol{\Psi}(\mathbf{x},\boldsymbol{\theta})}{\partial\boldsymbol{\theta}_1}\\ \vdots \\ \frac{\partial\boldsymbol{\Psi}(\mathbf{x},\boldsymbol{\theta})}{\partial\boldsymbol{\theta}_L}\end{bmatrix}^{\mathbf{T}} \\ &\quad\times\begin{bmatrix}\frac{\boldsymbol{\Theta}_1^{-1}}{\lambda_1} & & \\ & \ddots & \\ & & \frac{\boldsymbol{\Theta}_L^{-1}}{\lambda_L}\end{bmatrix}\begin{bmatrix}\frac{\partial\boldsymbol{\Psi}(\mathbf{x},\boldsymbol{\theta})}{\partial\boldsymbol{\theta}_1}\\ \vdots \\ \frac{\partial\boldsymbol{\Psi}(\mathbf{x},\boldsymbol{\theta})}{\partial\boldsymbol{\theta}_L}\end{bmatrix}\Delta\mathbf{y} \\ &= E(\boldsymbol{\theta}) - \varepsilon\boldsymbol{\Delta}\mathbf{y}^{\mathbf{T}}\left(\sum_{i=1}^{L}\frac{1}{\lambda_i}\left(\frac{\partial\boldsymbol{\Psi}(\mathbf{x},\boldsymbol{\theta})}{\partial\boldsymbol{\theta}_i}\right)^{\mathbf{T}}\right. \\ &\quad\left.\times\ \boldsymbol{\Theta}_i^{-1}\frac{\partial\boldsymbol{\Psi}(\mathbf{x},\boldsymbol{\theta})}{\partial\boldsymbol{\theta}_i}\right)\boldsymbol{\Delta}\mathbf{y} \\ &= E(\boldsymbol{\theta}) - \boldsymbol{\Delta}\mathbf{y}^{\mathbf{T}}\left(\sum_{i=1}^{L}\frac{\varepsilon}{\lambda_i}\mathbf{p}_i^{\mathbf{T}}\boldsymbol{\Theta}_i^{-1}\mathbf{p}_i\mathbf{q}_i\mathbf{q}_i^{\mathbf{T}}\right)\boldsymbol{\Delta}\mathbf{y} \\ &= E(\boldsymbol{\theta}) - \mu\boldsymbol{\Delta}\mathbf{y}^{\mathbf{T}}\left(\sum_{i=1}^{L}\mathbf{p}_i^{\mathbf{T}}\mathbf{p}_i\mathbf{q}_i\mathbf{q}_i^{\mathbf{T}}\right)\boldsymbol{\Delta}\mathbf{y} \\ &\cong E(\boldsymbol{\theta}+\delta\boldsymbol{\theta}^{\mathbf{GRD}}). \end{aligned} \tag{47}$$

This completes the proof of (41). Equation (42) is a straightforward result of substituting (43), (46), and (40) into (9). This completes the proof.

Remark 1: Note that (41) is equivalent to

$$E(\boldsymbol{\theta}+\delta\boldsymbol{\theta}^{\mathbf{PIL}})-E(\boldsymbol{\theta})\cong E(\boldsymbol{\theta}+\delta\boldsymbol{\theta}^{\mathbf{GRD}})-E(\boldsymbol{\theta}) \tag{48}$$

which implies that the quantity of the error-reduction in terms of the cost function (35) are approximately the same for the two algorithms for a given training pattern, despite that the learning algorithms differ. It is for this reason that we say $\delta\boldsymbol{\theta}^{\mathbf{PIL}}$ is *normalized* to the conventional gradient–descent learning algorithm. Also, we note that the normalization given by (40) is achieved neuron-by-neuron. The benefit from this fact is that, most likely, no retuning of the learning rate is needed when replacing the conventional online gradient–descent learning algorithm with the PIL algorithm, since, as will be further confirmed in a numerical study on the PIL algorithm for MLP in Section V, a learning rate which works well for the conventional online backpropagation learning algorithm also works well for the PIL algorithm of (42) (and vice versa).

Remark 2: The conventional online gradient–descent learning algorithm, on the other hand, can be derived from the general PIL algorithm (on the parameter level). In fact, we can use the following index:

$$\tilde{J}^{\mathrm{pres}}=\frac{1}{2\mu}\boldsymbol{\theta}^{\mathbf{T}}\boldsymbol{\theta} \tag{49}$$

as the measure of the preservation performance in the general PIL problem. In this case, the measure of the preservation performance is against the "deformation" of the parameters instead of against the deformation of the neurons. This is the essential difference between the two parameter update algorithms. If we then choose

$$\Xi=\left(\mathbf{I}-\mu\left(\frac{\partial\boldsymbol{\Psi}(\mathbf{x},\boldsymbol{\theta})}{\partial\boldsymbol{\theta}}\right)^{\mathbf{T}}\frac{\partial\boldsymbol{\Psi}(\mathbf{x},\boldsymbol{\theta})}{\partial\boldsymbol{\theta}}\right)^{-1} \tag{50}$$

the solution to the general PIL problem turns out to be

$$\delta\boldsymbol{\theta}_*^{\mathbf{PIL}}=-\mu\frac{\partial\boldsymbol{\Psi}(\mathbf{x},\boldsymbol{\theta})}{\partial\boldsymbol{\theta}}\Delta\mathbf{y} \tag{51}$$

which is precisely the same as the conventional online gradient–descent learning algorithm. We have Theorem 4 to further clarify the essential difference between the PIL algorithm and the conventional online gradient–descent learning algorithm.

Theorem 4: The following holds:

$$\|\delta\boldsymbol{\theta}_i^{\mathbf{GRD}}\|\leqslant\|\delta\boldsymbol{\theta}_i^{\mathbf{PIL}}\|,\qquad i=1,2,\ldots,L \tag{52}$$

$$\|\delta\boldsymbol{\theta}_i^{\mathbf{PIL}}\|_{\boldsymbol{\Theta}_i}\leqslant\|\delta\boldsymbol{\theta}_i^{\mathbf{GRD}}\|_{\boldsymbol{\Theta}_i},\qquad i=1,2,\ldots,L \tag{53}$$

where

$$\delta\boldsymbol{\theta}_i^{\mathbf{GRD}}=-\mu\mathbf{p}_i\mathbf{q}_i^{\mathbf{T}}\boldsymbol{\Delta}\mathbf{y},\qquad i=1,2,\ldots,L \tag{54}$$

is the neuron-wise form of (34); $\delta\boldsymbol{\theta}_i^{\mathbf{PIL}}$ is given by (42).

Proof: Note that for any square positive-definite matrix $\mathbf{A}$, inequalities (55)-(56), which are direct results of using the singular value decomposition for $\mathbf{A}$ followed by applying the Cauchy–Schwarz inequality, are well-known:

$$\frac{\mathbf{x}^{\mathbf{T}}\mathbf{A}^2\mathbf{x}}{\mathbf{x}^{\mathbf{T}}\mathbf{x}}\geqslant\left(\frac{\mathbf{x}^{\mathbf{T}}\mathbf{A}\mathbf{x}}{\mathbf{x}^{\mathbf{T}}\mathbf{x}}\right)^2 \tag{55}$$

and

$$\frac{\mathbf{x}^{\mathbf{T}}\mathbf{A}\mathbf{x}}{\mathbf{x}^{\mathbf{T}}\mathbf{x}}\frac{\mathbf{x}^{\mathbf{T}}\mathbf{A}^{-1}\mathbf{x}}{\mathbf{x}^{\mathbf{T}}\mathbf{x}}\geqslant 1 \tag{56}$$

where $\mathbf{x}$ is any nonzero vector with compatible dimension. Then

$$\begin{aligned}\frac{\|\delta\boldsymbol{\theta}_i^{\mathbf{PIL}}\|^2}{\|\delta\boldsymbol{\theta}_i^{\mathbf{GRD}}\|^2}&=\frac{\left(\delta\boldsymbol{\theta}_i^{\mathbf{PIL}}\right)^{\mathbf{T}}\delta\boldsymbol{\theta}_i^{\mathbf{PIL}}}{\left(\delta\boldsymbol{\theta}_i^{\mathbf{GRD}}\right)^{\mathbf{T}}\delta\boldsymbol{\theta}_i^{\mathbf{GRD}}}\\&=\frac{\mu^2\left(\frac{\mathbf{p}_i^{\mathbf{T}}\mathbf{p}_i}{\mathbf{p}_i^{\mathbf{T}}\boldsymbol{\Theta}_i^{-1}\mathbf{p}_i}\right)^2\mathbf{p}_i^{\mathbf{T}}\boldsymbol{\Theta}_i^{-2}\mathbf{p}_i\left(\mathbf{q}_i^{\mathbf{T}}\boldsymbol{\Delta}\mathbf{y}\right)^2}{\mu^2\mathbf{p}_i^{\mathbf{T}}\mathbf{p}_i\left(\mathbf{q}_i^{\mathbf{T}}\boldsymbol{\Delta}\mathbf{y}\right)^2}\\&=\frac{\frac{\mathbf{p}_i^{\mathbf{T}}\left(\boldsymbol{\Theta}_i^{-1}\right)^2\mathbf{p}}{\mathbf{p}_i^{\mathbf{T}}\mathbf{p}_i}}{\left(\frac{\mathbf{p}_i^{\mathbf{T}}\boldsymbol{\Theta}_i^{-1}\mathbf{p}_i}{\mathbf{p}_i^{\mathbf{T}}\mathbf{p}_i}\right)^2}\geqslant 1\end{aligned} \tag{57}$$

and

$$\begin{aligned}\frac{\|\delta\boldsymbol{\theta}_i^{\mathbf{PIL}}\|_{\boldsymbol{\Theta}_i}^2}{\|\delta\boldsymbol{\theta}_i^{\mathbf{GRD}}\|_{\boldsymbol{\Theta}_i}^2}&=\frac{\left(\delta\boldsymbol{\theta}_i^{\mathbf{PIL}}\right)^{\mathbf{T}}\boldsymbol{\Theta}_i\delta\boldsymbol{\theta}_i^{\mathbf{PIL}}}{\left(\delta\boldsymbol{\theta}_i^{\mathbf{GRD}}\right)^{\mathbf{T}}\boldsymbol{\Theta}_i\delta\boldsymbol{\theta}_i^{\mathbf{GRD}}}\\&=\frac{\mu^2\left(\frac{\mathbf{p}_i^{\mathbf{T}}\mathbf{p}_i}{\mathbf{p}_i^{\mathbf{T}}\boldsymbol{\Theta}_i^{-1}\mathbf{p}_i}\right)^2\mathbf{p}_i^{\mathbf{T}}\boldsymbol{\Theta}_i^{-1}\mathbf{p}_i\left(\mathbf{q}_i^{\mathbf{T}}\boldsymbol{\Delta}\mathbf{y}\right)^2}{\mu^2\mathbf{p}_i^{\mathbf{T}}\boldsymbol{\Theta}_i\mathbf{p}_i\left(\mathbf{q}_i^{\mathbf{T}}\boldsymbol{\Delta}\mathbf{y}\right)^2}\\&=\frac{\mathbf{p}_i^{\mathbf{T}}\mathbf{p}_i}{\mathbf{p}_i^{\mathbf{T}}\boldsymbol{\Theta}_i^{-1}\mathbf{p}}\frac{\mathbf{p}_i^{\mathbf{T}}\mathbf{p}_i}{\mathbf{p}_i^{\mathbf{T}}\boldsymbol{\Theta}_i\mathbf{p}_i}\leq 1.\end{aligned} \tag{58}$$

This completes the proof.

Inequality (52) in Theorem 4 means that the conventional online gradient–descent learning algorithm requires less parameter increment for each of the neurons than that of the PIL algorithm, given that both algorithms achieve approximately the same amount of error reduction in terms of the cost function E given by (35). On the other hand, (53) means that the PIL algorithm achieves a better preservation performance than that of the conventional online gradient–descent learning algorithm in terms of neuron deformation. It is now clear that, in some sense, we have three levels of general PIL problems: The global level, which is the highest level but for which an analytical solution is hard to obtain; the parameter level, which is the lowest level and whose solution is basically the conventional online gradient–descent learning algorithm; and the neuron level (building block level), which is a midlevel algorithm and is the main concern in this paper.

Remark 3: It is interesting to note that the PIL algorithm belongs to a more general class of online gradient–descent learning algorithms of the following form:

$$\delta\boldsymbol{\theta}^{\mathbf{GRD}} \triangleq -\mu\Phi\frac{\partial E}{\partial\boldsymbol{\theta}} \tag{59}$$

where Φ is an appropriately chosen symmetric *scaling matrix* which is positive–definite and $\mu > 0$ is a scalar learning rate, sometimes associated with an annealing procedure to ensure the final convergence phase of the algorithm. As pointed out in [9], from a point of view of stochastic learning, typical of these algorithms are stochastic second-order methods where Φ is an approximation of the inversion of the Hessian matrix, and the natural gradient method [10] where Φ is the inverse of Fisher information matrix. With the selection of Ξ given by (39), the PIL expressed by (22) can be written as

$$\delta\boldsymbol{\theta} = -\varepsilon\boldsymbol{\Theta}_\lambda^{-1}(\boldsymbol{\theta})\frac{\partial\boldsymbol{\Psi}(\mathbf{x},\boldsymbol{\theta})}{\partial\boldsymbol{\theta}}\Delta\mathbf{y} = -\varepsilon\boldsymbol{\Theta}_\lambda^{-1}(\boldsymbol{\theta})\frac{\partial E}{\partial\boldsymbol{\theta}} \tag{60}$$

where $\boldsymbol{\Theta}_\lambda(\boldsymbol{\theta})$ is a positive–definite matrix (and so is its inverse $\boldsymbol{\Theta}_\lambda^{-1}(\boldsymbol{\theta})$) since all the $\boldsymbol{\Theta}_i$'s are positive–definite matrices by construction. This takes the form of (59) and is thus a general gradient–descent method. However, unlike other general gradient–descent methods where the issue of the inversion operation over a big matrix (with its dimension equal to the number of all parameters in the network) has to be addressed, either by directly inverting the matrix, which requires $O(n^3)$ operations, or through some approximation approaches like the one proposed in [12] for the NGD algorithm where the number of operations is reduced to $O(n^2)$, the matrix $\boldsymbol{\Theta}_\lambda(\boldsymbol{\theta})$ is neuron-wise block-diagonal, i.e., a block-diagonal matrix with each of the blocks corresponding to the parameter vector of a neuron, and thus the computational burden associated with the inversion operation is greatly reduced. In fact, as will be seen in Section IV, there is no need at all to do matrix inversion for the PIL algorithm for the multilayer perceptron (MLP).

IV. PIL Algorithm For MLP

In this section, the general PIL algorithm is applied to the derivation of the learning algorithm for MLP. Since the general PIL algorithm is expressed in the neuron-wise form, the following discussions in this section will be based on a representative neuron, i.e., a neuron that can be any of the neurons in the neural network.

The nonlinear activation function of the neural network is chosen to be the hyperbolic tangent function

$$h(s) = \frac{1-e^{-s}}{1+e^{-s}}. \tag{61}$$

In the MLP, the input to the activation function of a neuron is given by

$$s = \bar{\mathbf{u}}^{\mathbf{T}}\boldsymbol{\theta} \tag{62}$$

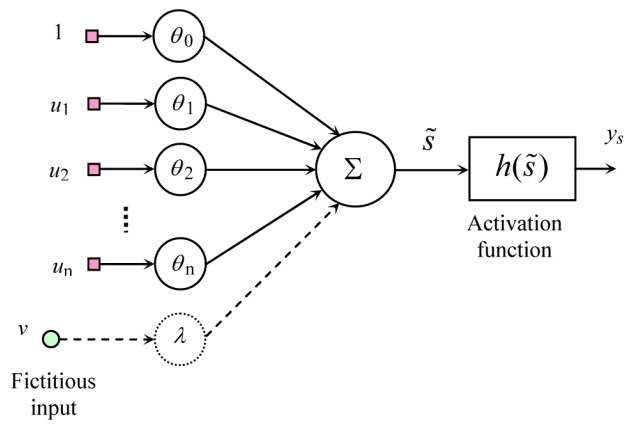

Fig. 1. Model of a representative neuron augmented with fictitious input.

where $\bar{\mathbf{u}} = [\mathbf{1}, \mathbf{u}^{\mathbf{T}}]^{\mathbf{T}}; \mathbf{u} = [u_1 \ u_2 \ \cdots \ u_n]^{\mathbf{T}} \in \Re^n$ is the input vector of the neuron and $\boldsymbol{\theta} = [\theta_0 \ \theta_1 \ \cdots \ \theta_n]^{\mathbf{T}}$ is the weight vector of this neuron. With the given activation function, a neuron in the MLP is expressed as

$$y_s = g(\mathbf{u},\boldsymbol{\theta}) = h(s) \tag{63}$$

where $y_s \in \Re$ is the output of the neuron. Then, we have

$$\mathbf{p} = \frac{\partial g(\mathbf{u},\boldsymbol{\theta})}{\partial\boldsymbol{\theta}} = h'(s)\bar{\mathbf{u}} \tag{64}$$

where

$$h'(s) = \frac{1}{2}(1-h^2(s)) = \frac{2e^{-s}}{(1+e^{-s})^2} \tag{65}$$

then

$$\boldsymbol{\Theta}(\boldsymbol{\theta}) = \int_{\mathbf{u}\in\mathcal{D}} \mathbf{p}(\mathbf{u},\boldsymbol{\theta})\mathbf{p}^T(\mathbf{u},\boldsymbol{\theta})d\mathbf{u} = \int_{\mathbf{u}\in\mathcal{D}} (h'(s))^2\bar{\mathbf{u}}\bar{\mathbf{u}}^{\mathbf{T}}d\mathbf{u}. \tag{66}$$

The key issue in obtaining an analytical solution to the PIL problem is to obtain a "closed form" formula to the integral (66) with a reasonably defined region $\mathcal{D}$. Unfortunately, this is generally not possible for this case. To overcome this difficulty, an extra fictitious input to the neuron is introduced, as is shown in Fig. 1. This amounts to rewriting (63) as

$$y_s = g(\tilde{\mathbf{u}},\tilde{\boldsymbol{\theta}}) = h(\tilde{s}) \tag{67}$$

where

$$\tilde{s} = \hat{\mathbf{u}}^{\mathbf{T}}\tilde{\boldsymbol{\theta}} \tag{68}$$

and $\hat{\mathbf{u}} = [1, \tilde{\mathbf{u}}^{\mathbf{T}}]^{\mathbf{T}}$; $\tilde{\mathbf{u}} = [\mathbf{u}^{\mathbf{T}}, v]^{\mathbf{T}}$ is the augmented input vector to the neuron; v denotes the fictitious input; and $\tilde{\boldsymbol{\theta}} = [\boldsymbol{\theta}^{\mathbf{T}}, \lambda]^{\mathbf{T}}$

with λ being the weight corresponding to the fictitious input. Note that for the augmented neuron

$$\tilde{\mathbf{p}} = \frac{\partial g(\tilde{\mathbf{u}}, \tilde{\boldsymbol{\theta}})}{\partial \tilde{\boldsymbol{\theta}}} = h'(\tilde{s}) \begin{bmatrix} 1 \\ \tilde{\mathbf{u}} \end{bmatrix}. \tag{69}$$

Equation (66) is accordingly extended to

$$\tilde{\boldsymbol{\Theta}}(\boldsymbol{\theta}, \lambda) = \int_{\tilde{\mathbf{u}} \in \tilde{\mathcal{D}}} (h'(\tilde{s}))^2 \begin{bmatrix} 1 \\ \tilde{\mathbf{u}} \end{bmatrix} [1 \quad \tilde{\mathbf{u}}^{\mathbf{T}}] \, d\tilde{\mathbf{u}}. \tag{70}$$

Now define the integral region as

$$\mathcal{D} = \prod_{i=1}^{n} [-b_i/2, b_i/2], \qquad b_i > 0 \text{ for } i = 1, 2, \ldots, n \tag{71}$$

and

$$\tilde{\mathcal{D}} = \mathcal{D} \times (-\infty, +\infty). \tag{72}$$

Then, the PIL algorithm for MLP can be obtained as

$$\delta \boldsymbol{\theta}^{\mathbf{PIL}} = -\mu h'(s) \frac{1 + \sum_{i=1}^{n} u_i^2}{1 + 12 \sum_{i=1}^{n} \frac{u_i^2}{b_i^2} + \frac{s^2}{\tau}} \times \begin{bmatrix} \frac{\theta_0 s}{\tau} + 1 \\ \frac{\theta_1 s}{\tau} + \frac{12}{b_1^2} u_1 \\ \vdots \\ \frac{\theta_n s}{\tau} + \frac{12}{b_n^2} u_n \end{bmatrix} (\mathbf{q}^{\mathbf{T}} \boldsymbol{\Delta} \mathbf{y}) \tag{73}$$

where $\tau = (\pi^2 - 6/3) \cong 1.29$. The derivation procedure is given in the Appendix. Note that in some applications where a linear output neuron rather than a nonlinear activation function is preferred, i.e., $g(s) = s$, it is straightforward to derive the PIL algorithm for those linear neurons, as given by

$$\delta \boldsymbol{\theta}^{\mathbf{PIL}} = -\mu \frac{1 + \sum_{i=1}^{n} u_i^2}{1 + \sum_{i=1}^{n} \frac{12}{b_1^2} u_i^2} \begin{bmatrix} 1 \\ \frac{12}{b_1^2} u_1 \\ \vdots \\ \frac{12}{b_n^2} u_n \end{bmatrix} (\mathbf{q}_\ell^{\mathbf{T}} \boldsymbol{\Delta} \mathbf{y}) \tag{74}$$

where $\mathbf{q}_\ell = [1 \ \cdots \ 1]_{1 \times m}^{\mathbf{T}}$. It is noted that for the linear neurons there would be no appreciable difference in using either the aforementioned PIL algorithm or the conventional online gradient–descent learning algorithm.

As a common selection, the integral region $\mathcal{D}$ is set by $b_i = 2$ for all $i = 1, 2, \ldots, n$. The reasons for so doing are as follows: Firs, this selection is apparently reasonable for those neurons whose inputs are the outputs of other neurons from its preceding layer, since the output range of the neurons in the network is $(-1, +1)$; and second, for those neurons which receive the network inputs (e.g., the neuron is in the first hidden layer), the range of input values can be readily normalized to $[-1, +1]$, as is a common practice in neural network training.

Remark 4: The calculation of the $\mathbf{q}^{\mathbf{T}} \boldsymbol{\Delta} \mathbf{y}$ in (73) involves a well-known error-backpropagation procedure, thus the PIL algorithm can be considered as a variation of the standard online BP algorithm. In fact, if we write the conventional BP learning algorithm in the following neuron-wise form:

$$\delta \boldsymbol{\theta}^{\mathbf{BP}} = -\mu h'(s) \begin{bmatrix} 1 \\ u_1 \\ \vdots \\ u_n \end{bmatrix} (\mathbf{q}^{\mathbf{T}} \boldsymbol{\Delta} \mathbf{y}) \tag{75}$$

then the PIL algorithm seems to be only a "minor" modification of the conventional online gradient–descent learning algorithm. We further note from (73) that unlike other general gradient–descent learning algorithms, the PIL algorithm requires no matrix inversion at all.

Remark 5: To gain some insights to the PIL algorithm for MLP, it is interesting to make a comparison between the parameter update laws for the two algorithms, PIL and natural gradient descent (NGD). The NGD algorithm is a recently developed stochastic learning algorithm, and is shown to be superior to the conventional gradient descent in that it has a much better chance of getting rid of the plateau area in the learning process [11]. It is noted, however, that one of the obstacles in applying the NGD algorithm to the feedforward neural networks is the lack of an analytical solution for the Riemann metric tensor matrix. To overcome the difficulty, Yang and Amari [11] proposed an adaptive method of directly obtaining an approximation of the inverse of the Fisher information matrix by applying the Kalman filter technique; and the computational cost is reduced to the order of $O(n^2)$ operations, thereby, instead of the $O(n^3)$, if the Fisher information matrix has to be numerically inverted. In [13], a variable learning rate scheme was proposed to further improve the algorithm. The only analytical solution of the NGD algorithm for neural networks obtained so far, to the best knowledge of the authors, is the one presented in [10] for a simple (nonlinear) perceptron, or the simplest MLP with one layer: a single, nonlinear neuron. It is interesting to make a comparison between the parameter update laws for the two algorithms, PIL and NGD, for this simple case, so that some insights can be gained.

Assume that the MLP has only one neuron with a hypertangent activation function $h(s)$ given by (61). The stochastic model of the perceptron considered in [10] is given by

$$y = h(s) + \xi \tag{76}$$

where

$$s = \boldsymbol{\theta}^{\mathbf{T}} \mathbf{x} \tag{77}$$

is the scalar input to the activation function; and $\xi \sim N(0, \sigma^2)$ is the Gaussian additive noise with the variance σ^2. It is noted

that there is no input bias term in this perceptron. It is further assumed that the input variable has a Gaussian distribution with identity variance matrix, i.e., $\mathbf{x} \sim N(0, \mathbf{I})$.

Under the aforementioned conditions, Amari [10] proved that, for the training input–output pattern $(\mathbf{x}, y)$, the parameter update law of the NGD algorithm can be explicitly expressed as

$$\delta\boldsymbol{\theta}^{\mathbf{NGD}} = -\mu(\kappa_1(\boldsymbol{\theta})\mathbf{x} + \kappa_2(\boldsymbol{\theta})s\boldsymbol{\theta})h'(s)\Delta y \tag{78}$$

where $\kappa_1(\boldsymbol{\theta})$ and $\kappa_2(\boldsymbol{\theta})$ are some known scalar functions of the parameter vector $\boldsymbol{\theta}$; $\Delta y = y - h(\boldsymbol{\theta}^{\mathbf{T}}\mathbf{x})$ is the prediction error for the current training pattern; and μ is the learning rate.

On the other hand, from (73), the parameter update law of the PIL algorithm for this case with all the b_i's equal to some given constant can be written as

$$\delta\boldsymbol{\theta}^{\mathbf{PIL}} = -\mu(\tau_1(\boldsymbol{\theta}, \mathbf{x})\mathbf{x} + \tau_2(\boldsymbol{\theta}, \mathbf{x})s\boldsymbol{\theta})h'(s)\Delta y \tag{79}$$

where $\tau_1(\boldsymbol{\theta}, \mathbf{x})$ and $\tau_2(\boldsymbol{\theta}, \mathbf{x})$ are some known scalar functions of the parameter vector $\boldsymbol{\theta}$ and input vector $\mathbf{x}$.

By comparing the parameter update laws of the NGD algorithm (78) and the PIL algorithm (79), it can be observed that they share a common feature, which is remarkably different from the standard BP algorithm, in that the parameter vector incremental direction is a linear combination of the input vector $\mathbf{x}$ and the parameter vector $\boldsymbol{\theta}$, with the latter being proportional to the scalar input to the activation function of the perceptron [i.e., the s given by (77)]. While for the standard BP algorithm, the parameter vector $\boldsymbol{\theta}$ increases along precisely the direction of the input vector $\mathbf{x}$, i.e.,

$$\delta\boldsymbol{\theta}^{\mathbf{BP}} = -\mu\mathbf{x}h'(s)\Delta y. \tag{80}$$

This observation reveals the intrinsic feature of the PIL and NGD algorithms that distinguishes them from the standard BP algorithm.

Finally, it is also worth noting that it is straightforward to verify that the parameter update law (73) remains exactly the same if, instead of the hyperbolic tangent function, a logistic activation function $h(s) = (1/1 + e^{-s})$ is used as the activation function.

V. Numerical Experiments

The PIL algorithm was tested on various benchmark problems. Due to space limitations, results on three benchmark problems are presented here, with additional results available in [14]. The computer simulations are carried out mainly by comparing the performance of the PIL algorithm with the standard online BP algorithm and SDLM method, which is a well-known simplified second-order stochastic learning algorithm for feedforward neural networks developed in [1]. The comparison will mainly focus on the performance of convergence speed and/or the final approximation error. Other issues such as generalization and adaptation to time-varying underlying functions are not addressed here, since they are affected not only by the parameter update laws, but also by some other factors such as training process control (e.g., early-stopping), training data selection, structural adaptation (e.g., neuron pruning/recruiting), etc.

Briefly, the SDLM algorithm is obtained by modifying the well-known LM optimization algorithm in three ways: First, the stochastic learning principle is applied to the standard LM algorithm by updating the network parameters via the LM algorithm once every single training pattern is presented; second, it uses only the diagonal terms of the approximated Hessian matrix, thus significantly reducing the computational cost; third, it uses a first-order filter to smooth out the noisy diagonal terms of the approximated Hessian matrix. The SDLM is an efficient learning algorithm which usually outperforms the standard BP algorithm. More importantly, to the best knowledge of the authors, the SDLM is perhaps the simplest stochastic "second-order" algorithm since its computational cost is the same order as the standard BP algorithm, i.e., it requires only $O(n)$ operations in each iteration. It is noted that, unlike the conventional BP and the PIL algorithms where only the learning rate must be tuned, there are three parameters in the SDLM algorithm, i.e., the global learning rate, a small constant ζ that is used to prevent the algorithm from blowing up in case the second derivative is close to zero, and a filter constant γ that must be simultaneously tuned. Details of this algorithm can be found in [1].

In the computer simulations, we follow some of the conventions in MLP training (see e.g., [1]). These include the following.

1) The weights to each of the neurons are initialized with uniformly distributed random values in $[-1/m, 1/m]$ where m is the fan-in (i.e., number of inputs) to the neuron.
2) For each training epoch, the data is shuffled (randomly permuted) before it is used for training.
3) The input data range is normalized to be in the interval $[-1, +1]$.

Also, the following conditions were imposed in the numerical study.

1) All of the MLPs used had one hidden layer with hyper-tangent sigmoid activation function.
2) For each of the learning tasks, three MLP neural networks with exactly the same structure (i.e., the same number of hidden neurons) but using different learning algorithms (PIL versus standard online BP and SDLM), were run in parallel with the same initial conditions (i.e., initial weights). This means that all three networks receive the same sequence of input–output patterns during the training process.
3) The learning process was visualized by the *learning curve*, i.e., the plot of mean square error (MSE) versus number-of-epochs.
4) The output layer was chosen to be nonlinear (with the same activation function as for the hidden neurons) for the classification task, and linear for the function approximation task.
5) For the classification problem, the popular 40–20–40 criterion (see e.g., [3]) was used: An output was considered to be a logical zero if it was in the lower 40% of the output range, a one if it was in the upper 40%, and indeterminate (and, therefore, incorrect) if it was in the middle 20% of the range. The desired value logic one (zero) was chosen to be 90% (10%) of the maximum (minimum) output value. Since for an output neuron, the output range was

TABLE I
MSE OF THE TRAINED NEURAL NETWORKS

	1	2	3	4	5	6	7	8	9	1 0
P I L	0.29	0.24	0.31	0.29	0.34	0.37	0.29	0.44	0.24	0.34
B P	0.39	0.72	0.28	0.23	0.52	0.23	0.32	0.72	0.72	0.32
SDLM	0.50	0.63	0.24	0.51	0.50	0.45	0.45	0.75	0.35	0.75

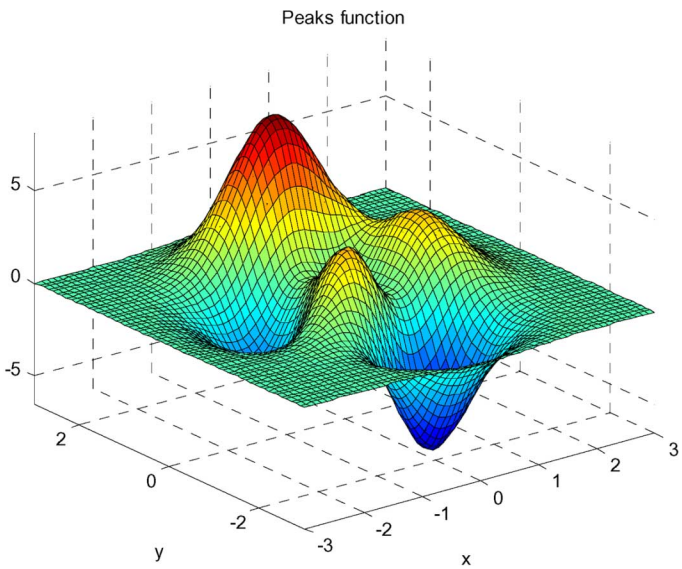

Fig. 2. Peaks function.

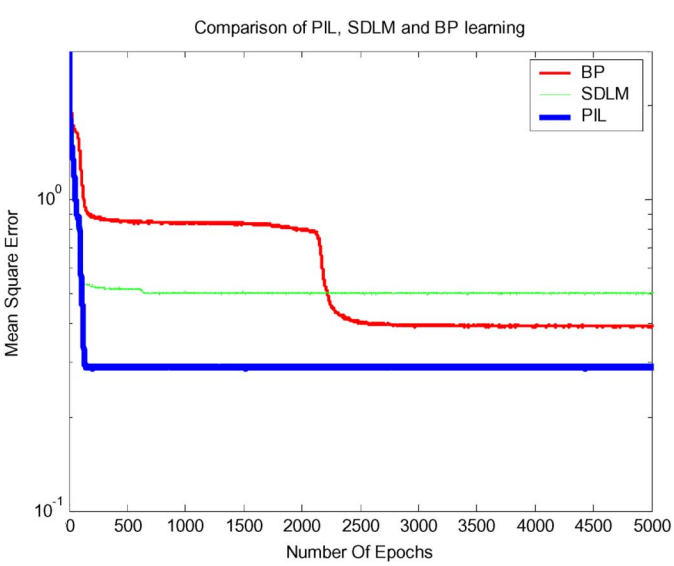

Fig. 3. Typical learning curves for Peaks function.

$(-1, 1)$, the desired logic one (zero) value was, therefore, $0.8(-0.8)$.

Case 1: Peaks function approximation

The task was to approximate the following MATLAB® Peaks function using MLP:

$$z = 3(1-x)^2 e^{(-x^2-(y+1)^2)} - 10\left(\frac{x}{5} - x^3 - y^5\right) e^{(-x^2-y^2)} - \frac{1}{3} e^{(-(x+1)^2-y^2)}. \quad (81)$$

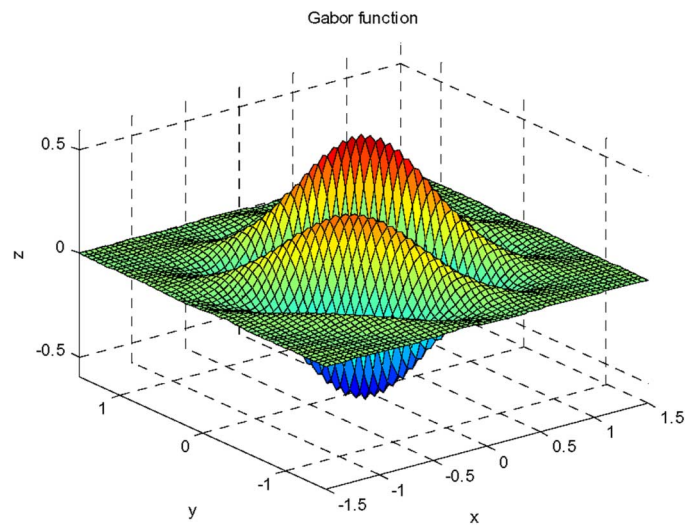

Fig. 4. Gabor function.

The function mainly consists of a couple of peaks as shown in Fig. 2. The training data consisted of 3721 input–output data points which were obtained by evaluating the function (81) over a 61×61 uniformly distributed grid on the square $[-3, +3] \times [-3, +3]$ in $x - y$ plane. Ten hidden neurons were used. The learning rate was set to 0.003 for all three algorithms, which is found to be well tuned. It was observed in this simulation study, as was anticipated from the discussion in Remark 1 of Section III, that a learning rate which works well for the conventional online backpropagation learning algorithm also works well for the PIL algorithm, and vice versa. The other two parameters for the SDLM algorithm were tuned to $\zeta = 0.01$ and $\gamma = 0.000\,01$, respectively, which gave satisfactory performance. Ten numerical simulations were conducted, with each starting from different, randomly generated initial weights. The number of training epochs was set to 5000, which was sufficient for each of the training algorithms to reach steady-state MSE values in all ten runs. Table I shows the MSE values of the three trained neural networks for all ten runs.

By comparing the three MSE values obtained by the three different training algorithms for each of the ten runs, it is seen that the PIL algorithm "won the competition" among the three algorithms six times out of the ten runs, in terms of achieving the smallest MSE values, while the BP won three times and the SDLM won one time, clearly indicating that the PIL algorithm has a higher chance of achieving a better solution. It is also observed that the PIL algorithm gives better convergence speed than the conventional online BP learning algorithm. Fig. 3 shows a typical learning process in which the standard BP algorithm became stuck in the so-called *flat area* or *plateau*, which is common in the conventional BP algorithm and is one of the

TABLE II
MSE ($\times 10^2$) OF THE TRAINED NEURAL NETWORKS

	1	2	3	4	5	6	7	8	9	10
PIL	0.26	1.72	0.47	1.72	0.22	0.40	0.22	0.37	0.18	0.18
SDLM	0.86	0.18	0.89	0.8	0.26	0.12	0.86	0.87	0.32	0.38

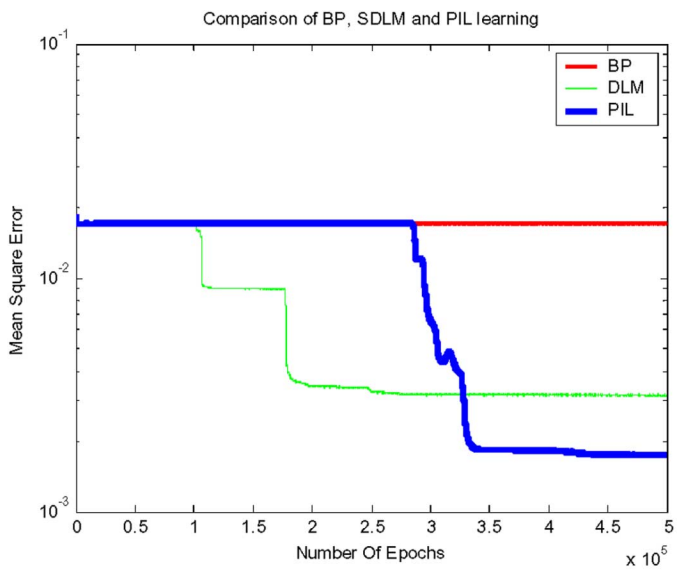

Fig. 5. Typical learning curves for Gabor function.

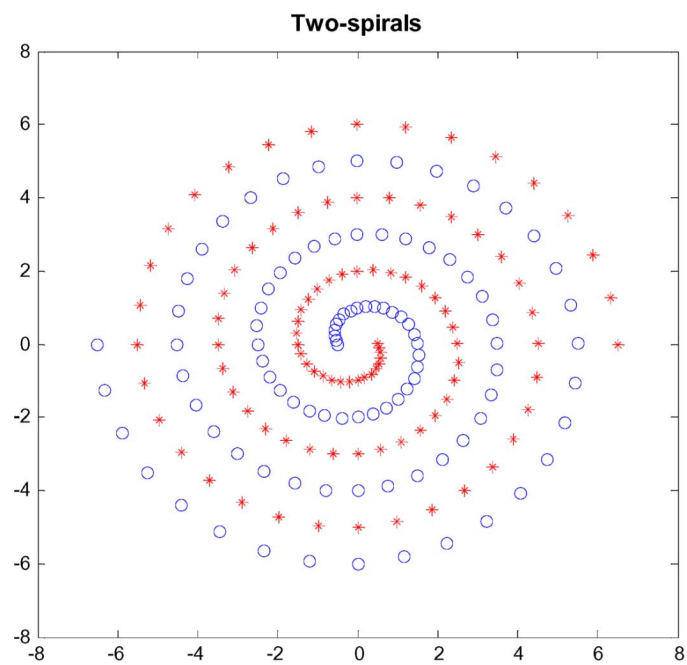

Fig. 6. Two spirals.

major reasons for the BP being criticized as slow in convergence. In nine out of the ten runs performed for this study, the BP algorithm became stuck in a plateau of some duration, while the PIL and the SDLM algorithms did not. It was also observed that the SDLM algorithm and the PIL algorithm had similar convergence speeds in reaching their own steady-state error values.

Case 2: Gabor function approximation

The task was to approximate the following 2-D Gabor function using MLP, shown in Fig. 4:

$$z = \frac{2}{\pi} e^{(-2(x^2+y^2))} \sin(2\pi(x+y)). \tag{82}$$

The training data consisted of 3721 input–output data points which were obtained by evaluating the function (82) over a 61×61 uniformly distributed grid on the square $[-1.5, +1.5] \times [-1.5, +1.5]$ in $x-y$ plane. Ten hidden neurons were used for all three neural networks. All three algorithms were found to be well-tuned using a learning rate of 0.001. The other two parameters for the SDLM algorithm were tuned to $\zeta = 0.01$ and $\gamma = 0.00001$, respectively, which gave satisfactory performance. Ten numerical simulations were conducted, with each starting from different, randomly generated initial weights. The number of training epochs was set to 5×10^5, which was shown to be sufficient for each of the training algorithms to reach steady-state MSE values (if they did not fail) in all ten runs. In this simulation study, the Gabor function approximation was shown to be a "hard problem" for standard BP learning since for all ten runs, the BP algorithm "completely failed," i.e., the MSE never dropped below its initial value of 1.72×10^{-2}. Fig. 5 shows a typical learning process. The PIL algorithm and the SDLM algorithm, on the other hand, did finally learn the function although they initially were stuck on a long plateau, except in simulation 2 and simulation 4, where the PIL algorithm failed within the specified number of learning epochs. Table II shows the MSE values of the two neural networks trained by PIL and SDLM algorithms for all ten runs. It is seen from the table that the PIL algorithm, even though it failed twice, won seven times out of the ten runs in terms of reaching a smaller MSE value, indicating that the PIL algorithm has a higher chance of achieving a better solution than the SDLM algorithm does.

Case 3: Two spirals classification

The task was to learn to discriminate between two sets of training points which lie on two distinct spirals in the $x-y$ plane, as is shown in Fig. 6. These spirals coil three times around the origin and around one another. This appears to be a very difficult classification task for the standard MLP. The standard backpropagation learning has been reported to fail completely when using an MLP with a single hidden layer [15]. A three-hidden layer MLP, with five neurons in each layer and shortcut connections between all the layers, was found to be able to solve this difficult problem. The two-spiral problem has since been popular in the neural network community and has been extensively used as a benchmark for evaluating the classification performance of various neural networks with different learning algorithms and/or structures.

The training data set consisted of 194 input–output pairs, available at [16]. Forty five hidden neurons were used for this task. All three algorithms were found to be well tuned

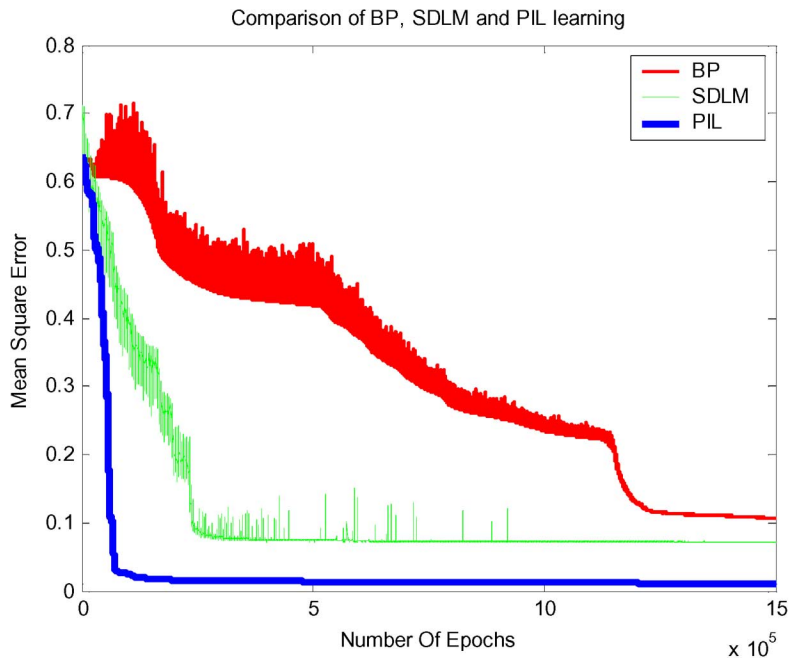

Fig. 7. Learning curves for two spirals.

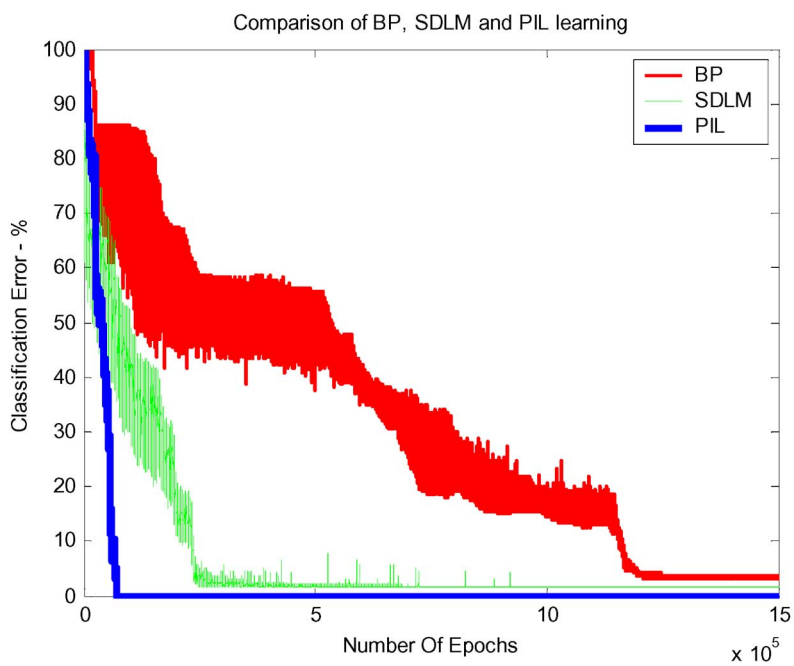

Fig. 8. Learning curves for two spirals.

using a learning rate of 0.004. The other two parameters for the SDLM algorithm were tuned to $\zeta = 0.1$ and $\gamma = 0.000\,01$, respectively, which gave satisfactory performance. The 1.5×10^6 training epochs were sufficient for each of the learning algorithms to reach steady-state classification errors. and Figs. 7 and 8 show the learning curves of MSE versus number of epochs and classification error versus number of epochs, respectively, for a typical run. The PIL algorithm learned the task with zero classification errors after around 7×10^4 epochs, while the SDLM algorithm and the standard online BP took much longer to reach their nonzero steady-state classification errors (1.54% and 3.93% for SDLM and BP algorithms, respectively). Thus, the PIL again exhibited a superior performance over the other two algorithms in terms of both classification accuracy and convergence speed. More than ten simulations were conducted using the same neural network structure but starting from different initial weights. All of the runs exhibited similar results: The PIL algorithm converged to its steady-state classification errors in less than 10^5 epochs while the other two algorithms took at least 2×10^5 epochs or more, as shown in Fig. 8.

VI. Conclusion

A new learning strategy for neural networks, called parameter incremental learning, has been presented. The essence of the PIL strategy is that in the process of adapting the network to training data by adjusting its parameters, the past learning of the network is also explicitly required to be preserved to a certain extent. Since the general PIL strategy is usually not computationally realizable at the global network level, the general PIL problem is proposed at the neuron level so the measure of preservation is against the deformation of each individual neuron instead of the network as a whole. For this suboptimal strategy, the analytical solution is presented, with the "free parameters" being determined by normalizing the error-reduction property of the algorithm to that of the standard gradient–descent algorithm. An alternative interpretation of the PIL algorithm at the neuron level, which reveals some insights to the algorithm, is also given. The PIL algorithm for MLP is then derived, based on the general PIL algorithm at the neuron level and the introduction of an extra fictitious input to the neuron. To gain further insight into the new algorithm, an analytical comparison is made between the parameter update laws of the PIL algorithm and the NGD algorithm for the simplest MLP, i.e., the MLP with multi-input, single output, and single nonlinear neuron. It is shown that the two algorithms share a strong similarity which distinguishes them from the standard BP algorithm. Numerical studies have been carried out by comparing the learning process of the PIL algorithm with that of the standard BP algorithm and the SDLM algorithm.

The following conclusion can be drawn from the benchmark studies and analysis.

1) For the numerical studies conducted, the PIL algorithm is remarkably superior to the standard online BP learning algorithm in terms of convergence speed. This is due to the fact that, compared to the standard BP algorithm, the PIL algorithm has a much better chance to avoid plateau areas.
2) The computation cost for the PIL algorithm and the standard BP algorithm are almost the same.
3) Similar to the BP algorithm, the PIL algorithm has only one parameter that must be tuned in applications. The learning rate that is well tuned for the standard BP algorithm is also well tuned for the PIL algorithm, and vice versa. This implies that the PIL can readily replace the standard online BP algorithm already in use to get better performance, even without retuning the learning rate.
4) Compared to the SDLM algorithm, the PIL algorithm has a high chance of finding a better solution. Moreover, the PIL algorithm is easier to use since the SDLM algorithm has three parameters, instead of one, that must be simultaneously tuned.

In general, it is expected that the PIL algorithm has the potential to replace the standard stochastic BP algorithm and become yet another standard online learning algorithm for MLP due to its desirable features. Particularly, the PIL algorithm is a strong candidate to solve difficult problems where the conventional BP is unable to find a satisfactory solution.

Appendix
Derivation of $\delta\boldsymbol{\theta}^{\mathbf{PIL}}$ for MLP

Note that by first integrating over the fictitious variable v the (70) can be written as

$$\tilde{\boldsymbol{\Theta}}(\boldsymbol{\theta},\lambda) = \int_{\mathbf{u}\in\mathcal{D}} \boldsymbol{\Theta}_v(\mathbf{u},\boldsymbol{\theta},\lambda)d\mathbf{u} \tag{83}$$

where

$$\boldsymbol{\Theta}_v(\mathbf{u},\boldsymbol{\theta},\lambda) \triangleq \int_{-\infty}^{+\infty} (g'(\tilde{s}))^2 \begin{bmatrix} 1 \\ \tilde{\mathbf{u}} \end{bmatrix} [1 \quad \tilde{\mathbf{u}}^{\mathbf{T}}]\, dv \tag{84}$$

and (84) can further be written as

$$\begin{aligned}\boldsymbol{\Theta}_v(\mathbf{u},\boldsymbol{\theta},\lambda) &= \int_{-\infty}^{+\infty} (g'(s+\lambda v))^2 \begin{bmatrix} 1 \\ \mathbf{u} \\ v \end{bmatrix} [1 \quad \mathbf{u}^{\mathbf{T}} \quad v]\, dv \\ &= \begin{bmatrix} \rho_1 & \rho_1\mathbf{u}^{\mathbf{T}} & \rho_2 \\ \rho_1\mathbf{u} & \rho_1\mathbf{u}\mathbf{u}^{\mathbf{T}} & \rho_2\mathbf{u} \\ \rho_2 & \rho_2\mathbf{u}^{\mathbf{T}} & \rho_3 \end{bmatrix}\end{aligned} \tag{85}$$

where

$$\begin{aligned}\rho_1 &\triangleq \int_{-\infty}^{+\infty} (g'(s+\lambda v))^2 dv \\ &= \int_{-\infty}^{+\infty} \left(\frac{2e^{-(s+\lambda v)}}{\left(1+e^{-(s+\lambda v)}\right)^2} \right)^2 dv \end{aligned} \tag{86}$$

$$\begin{aligned}\rho_2 &\triangleq \int_{-\infty}^{+\infty} (g'(s+\lambda v))^2 v dv \\ &= \int_{-\infty}^{+\infty} \left(\frac{2e^{-(s+\lambda v)}}{\left(1+e^{-(s+\lambda v)}\right)^2} \right)^2 v dv \end{aligned} \tag{87}$$

$$\begin{aligned}\rho_3 &\triangleq \int_{-\infty}^{+\infty} (g'(s+\lambda v))^2 v^2 dv \\ &= \int_{-\infty}^{+\infty} \left(\frac{2e^{-(s+\lambda v)}}{\left(1+e^{-(s+\lambda v)}\right)^2} \right)^2 v^2 dv. \end{aligned} \tag{88}$$

Solving the integral (84) basically amounts to finding the previous three integrals, which are found to be

$$\rho_1 = \frac{2}{3|\lambda|} \quad \rho_2 = -\frac{2s}{3\lambda^2} \quad \rho_3 = \frac{2}{3|\lambda|^3}(\tau + s^2) \tag{89}$$

where $\tau = (\pi^2 - 6/3) \cong 1.29$. Then, (90), as shown at the top of the next page, holds. It is straightforward to obtain the integrals (91)–(92), as shown at the top of the next page, contained in (90). Then, (90) becomes (93), as shown at the top of the next page. It is easy to verify that the inverse of $\tilde{\boldsymbol{\Theta}}(\boldsymbol{\theta},\lambda)$ is given by (94), as shown at the top of the next page. We further have

$$\tilde{\boldsymbol{\Theta}}^{-1}(\boldsymbol{\theta},\lambda)\tilde{\mathbf{p}} = \frac{3|\lambda|}{2\prod_{i=1}^{n} b_i} g'(\tilde{s}) \begin{bmatrix} 1 + \frac{\theta_0}{\tau}s + \frac{|\lambda|}{\tau}\theta_0 v \\ \frac{\theta_1}{\tau}s + \frac{12}{b_1^2}u_1 + \frac{|\lambda|}{\tau}\theta_1 v \\ \vdots \\ \frac{\theta_n}{\tau}s + \frac{12}{b_n^2}u_n + \frac{|\lambda|}{\tau}\theta_n v \\ \frac{|\lambda|}{\tau}s + \frac{\lambda^2}{\tau}v \end{bmatrix} \tag{95}$$

$$\tilde{\boldsymbol{\Theta}}(\boldsymbol{\theta},\lambda)=\int_{\mathbf{u}\in\mathcal{D}}\begin{bmatrix}\rho_1 & \rho_1\mathbf{u}^{\mathbf{T}} & \rho_2\\ \rho_1\mathbf{u} & \rho_1\mathbf{u}\mathbf{u}^{\mathbf{T}} & \rho_2\mathbf{u}\\ \rho_2 & \rho_2\mathbf{u}^{\mathbf{T}} & \rho_3\end{bmatrix}d\mathbf{u}=\begin{bmatrix}\frac{2}{3|\lambda|}\int_{\mathbf{u}\in\mathcal{D}}d\mathbf{u} & \frac{2}{3|\lambda|}\int_{\mathbf{u}\in\mathcal{D}}\mathbf{u}^{\mathbf{T}}d\mathbf{u} & -\frac{2}{3\lambda^2}\int_{\mathbf{u}\in\mathcal{D}}sd\mathbf{u}\\ \frac{2}{3|\lambda|}\int_{\mathbf{u}\in\mathcal{D}}\mathbf{u}d\mathbf{u} & \frac{2}{3|\lambda|}\int_{\mathbf{u}\in\mathcal{D}}\mathbf{u}\mathbf{u}^{\mathbf{T}}d\mathbf{u} & -\frac{2}{3\lambda^2}\int_{\mathbf{u}\in\mathcal{D}}s\mathbf{u}d\mathbf{u}\\ -\frac{2}{3\lambda^2}\int_{\mathbf{u}\in\mathcal{D}}sd\mathbf{u} & -\frac{2}{3\lambda^2}\int_{\mathbf{u}\in\mathcal{D}}s\mathbf{u}^{\mathbf{T}}d\mathbf{u} & \frac{2}{3|\lambda|^3}\left(\int_{\mathbf{u}\in\mathcal{D}}(\tau+s^2)d\mathbf{u}\right)\end{bmatrix} \tag{90}$$

$$\left.\begin{aligned}&\int_{\mathbf{u}\in\mathcal{D}}d\mathbf{u}=\textstyle\prod_{i=1}^n b_i,\qquad \int_{\mathbf{u}\in\mathcal{D}}\mathbf{u}d\mathbf{u}=0\\ &\int_{\mathbf{u}\in\mathcal{D}}sd\mathbf{u}=\theta_0\textstyle\prod_{i=1}^n b_i,\quad \int_{\mathbf{u}\in\mathcal{D}}s\mathbf{u}d\mathbf{u}=\frac{1}{12}\prod_{i=1}^n b_i\begin{bmatrix}\theta_1 b_1^2\\ \vdots\\ \theta_n b_n^2\end{bmatrix}\end{aligned}\right\} \tag{91}$$

$$\left.\begin{aligned}&\int_{\mathbf{u}\in\mathcal{D}}\mathbf{u}\mathbf{u}^{\mathbf{T}}d\mathbf{u}=\frac{1}{12}\prod_{i=1}^n b_i\begin{bmatrix}b_1^2 & & \mathrm{O}\\ & \ddots & \\ \mathrm{O} & & b_n^2\end{bmatrix}\\ &\int_{\mathbf{u}\in\mathcal{D}}\left(\tau+s^2\right)d\mathbf{u}=\prod_{i=1}^n b_i\left(\tau+\theta_0^2+\frac{1}{12}\sum_{i=1}^n\theta_i^2 b_i^2\right)\end{aligned}\right\} \tag{92}$$

$$\tilde{\boldsymbol{\Theta}}(\boldsymbol{\theta},\lambda)=\frac{2}{3|\lambda|}\prod_{i=1}^n b_i\begin{bmatrix}1 & 0 & \cdots & 0 & -\frac{\theta_0}{|\lambda|}\\ 0 & \frac{b_1^2}{12} & \cdots & 0 & -\frac{\theta_1 b_1^2}{12|\lambda|}\\ \vdots & \vdots & \ddots & \vdots & \vdots\\ 0 & 0 & \cdots & \frac{b_n^2}{12} & -\frac{\theta_n b_n^2}{12|\lambda|}\\ -\frac{\theta_0}{|\lambda|} & -\frac{\theta_1 b_1^2}{12|\lambda|} & \cdots & -\frac{\theta_n b_n^2}{12|\lambda|} & \frac{1}{\lambda^2}\left(\tau+\theta_0^2+\sum_{i=1}^n\frac{\theta_i^2 b_i^2}{12}\right)\end{bmatrix} \tag{93}$$

$$\tilde{\boldsymbol{\Theta}}^{-1}(\boldsymbol{\theta},\lambda)=\frac{3|\lambda|}{2\prod_{i=1}^n b_i}\begin{bmatrix}1+\frac{\theta_0^2}{\tau} & \frac{\theta_0\theta_1}{\tau} & \frac{\theta_0\theta_2}{\tau} & \cdots & \frac{\theta_0\theta_n}{\tau} & \frac{|\lambda|\theta_0}{\tau}\\ \frac{\theta_0\theta_1}{\tau} & \frac{12}{b_1^2}+\frac{\theta_1^2}{\tau} & \frac{\theta_1\theta_2}{\tau} & \cdots & \frac{\theta_1\theta_n}{\tau} & \frac{|\lambda|\theta_1}{\tau}\\ \frac{\theta_0\theta_2}{\tau} & \frac{\theta_1\theta_2}{\tau} & \frac{12}{b_3^2}+\frac{\theta_2^2}{\tau} & \cdots & \frac{\theta_2\theta_n}{\tau} & \frac{|\lambda|\theta_2}{\tau}\\ \vdots & \vdots & \vdots & \ddots & \vdots & \vdots\\ \frac{\theta_0\theta_n}{\tau} & \frac{\theta_1\theta_n}{\tau} & \frac{\theta_2\theta_n}{\tau} & \cdots & \frac{12}{b_n^2}+\frac{\theta_n^2}{\tau} & \frac{|\lambda|\theta_n}{\tau}\\ \frac{|\lambda|\theta_0}{\tau} & \frac{|\lambda|\theta_1}{\tau} & \frac{|\lambda|\theta_2}{\tau} & \cdots & \frac{|\lambda|\theta_n}{\tau} & \frac{\lambda^2}{\tau}\end{bmatrix} \tag{94}$$

and

$$\begin{aligned}&\tilde{\mathbf{p}}^{\mathbf{T}}\tilde{\boldsymbol{\Theta}}^{-1}(\boldsymbol{\theta},\lambda)\tilde{\mathbf{p}}\\ &\quad=\frac{3|\lambda|}{2\prod_{i=1}^n b_i}g'^2(\tilde{s})\\ &\qquad\times\left(1+12\sum_{i=1}^n\frac{u_i^2}{b_i^2}+\frac{s^2+2s|\lambda|v+\lambda^2v^2}{\tau}\right).\end{aligned} \tag{96}$$

Thus

$$\begin{aligned}\delta\tilde{\boldsymbol{\theta}}^{\mathrm{opt}}&=-\mu\frac{\tilde{\mathbf{p}}^{\mathbf{T}}\tilde{\mathbf{p}}}{\tilde{\mathbf{p}}^{\mathbf{T}}\boldsymbol{\Theta}^{-1}\tilde{\mathbf{p}}}\tilde{\boldsymbol{\Theta}}^{-1}\tilde{\mathbf{p}}(\mathbf{q}^{\mathbf{T}}\boldsymbol{\Delta}\mathbf{y})\\ &=-\mu\frac{g'(\tilde{s})\left(1+\sum_{i=1}^n u_i^2+v^2\right)}{\left(1+12\sum_{i=1}^n\frac{u_i^2}{b_i^2}+\frac{s^2+2s|\lambda|v+\lambda^2v^2}{\tau}\right)}\\ &\quad\times\begin{bmatrix}\frac{\theta_0 s}{\tau}+1+\frac{|\lambda|}{\tau}\theta_0 v\\ \frac{\theta_1 s}{\tau}+\frac{12}{b_1^2}u_1+\frac{|\lambda|}{\tau}\theta_1 v\\ \vdots\\ \frac{\theta_n s}{\tau}+\frac{12}{b_n^2}u_n+\frac{|\lambda|}{\tau}\theta_n v\\ \frac{|\lambda|}{\tau}s+\frac{\lambda^2}{\tau}v\end{bmatrix}(\mathbf{q}^{\mathbf{T}}\boldsymbol{\Delta}\mathbf{y}).\end{aligned} \tag{97}$$

Letting the fictitious input $v=0$ in (97) yields (73).

REFERENCES

[1] Y. LeCun, L. Bottou, G. B. Orr, and K.-R. Müller, "Efficient BackProp," in *Neural Networks: Tricks of the Trade*, G. B. Orr and K.-R. Müller, Eds. Berlin, Germany: Springer-Verlag, 1998, pp. 9–53.

[2] R. D. Reed and R. J. Marks II, *Neural Smithing: Supervised Learning in Feedforward Artificial Neural Networks*. Cambridge, MA: MIT Press, 1999.

[3] M. Riedmiller, "Advanced supervised learning in multi-layer perceptrons—From backpropagation to adaptive learning algorithms," *Int. J. Comput. Standards Interfaces*, vol. 16, no. 3, pp. 265–278, 1994, Special Issue on Neural Networks.

[4] R. Battiti, "First- and second-order methods for learning: Between steepest descent and Newton's method," *Neural Comput.*, vol. 4, no. 2, pp. 141–166, 1992.
[5] D. Chakraborty and N. R. Pal, "A novel training scheme for multilayered perceptrons to realize proper generalization and incremental learning," *IEEE Trans. Neural Netw.*, vol. 14, no. 1, pp. 1–14, Jan. 2003.
[6] R. Polikar, L. Udpa, S. S. Udpa, and V. , "Learn++: An incremental learning algorithm for supervised neural networks," *IEEE Trans. Syst., Man, Cybern. C, Appl. Rev.*, vol. 31, no. 4, pp. 497–508, Nov. 2001.
[7] J. Platt, "A resource-allocating network for function interpolation," *Neural Comput.*, vol. 3, no. 2, pp. 213–225, 1991.
[8] A. P. Engelbrecht and R. Brits, "A clustering approach to incremental learning for feedforward neural networks," *Proc. Int. Joint Conf. Neural Netw.*, vol. 3, pp. 2019–2024, 2001.
[9] L. Bottou, "Stochastic Learning," in *Advanced Lectures on Machine Learning*, ser. LNAI 3176, O. Bousquet and U. von Luxburg, Eds. Berlin, Germany: Springer-Verlag, 2004, pp. 146–168.
[10] S.-I. Amari, "Natural gradient works efficiently in learning," *Neural Comput.*, vol. 10, no. 2, pp. 251–276, 1998.
[11] H. H. Yang and S. Amari, "Complexity issues in natural gradient descent method for training multilayer perceptrons," *Neural Comput.*, vol. 10, no. 8, pp. 2137–2157, 1998.
[12] S. Amari, H. Park, and K. Fukumizu, "Adaptive method of realizing natural gradient learning for multilayer perceptrons," *Neural Comput.*, vol. 12, no. 6, pp. 1399–1409, 2000.
[13] W. Wan, "Implementing online natural gradient learning: Problems and solutions," *IEEE Trans. Neural Netw.*, vol. 17, no. 2, pp. 317–329, Mar. 2006.
[14] S. Wan, "Parameter Incremental Learning Algorithm for Neural Networks," Ph.D. dissertation, West Virginia Univ., Morgantown, WV, May 2005.
[15] K. J. Lang and M. J. Witbrock, "Learning to tell two spirals apart," in *Proceedings of the 1988 Connectionist Models Summer School*. San Mateo, CA: Morgan Kaufmann, 1988.
[16] Carnegie Mellon Univ. Pittsburgh, PA, 1993 [Online]. Available: http://www-2.cs.cmu.edu/afs/cs/project/ai-repository/ai/areas/neural/bench/cmu/

Sheng Wan received the B.S. degree in electrical engineering and the M.S. degree in control systems from Nanjing University of Aeronautics and Astronautics, Nanjing, China, in 1983 and 1986, respectively, and the Ph.D. degree in aerospace engineering from West Virginia University, Morgantown, in 2005.

He is currently with the GE Transportation, Erie, PA. His main research interests include dynamic systems, control theory and applications, neural networks learning algorithms, and biologically plausible neural networks.

Larry E. Banta (M'87–SM'90) received the B.A. degree in physics from Kalamazoo College, Kalamazoo, MI, in 1973, and the M.S. and Ph.D. degrees in mechanical engineering from the Georgia Institute of Technology, Atlanta, in 1981 and 1987, respectively.

From 1974 to 1976, he worked for Allen Testproducts as an R&D Technician and, from 1977 to 1985, for the Georgia Tech Research Institute as a Research Scientist. Since 1986, he has been a faculty member in the Mechanical and Aerospace Engineering Department, West Virginia University, Morgantown, where he is currently an Associate Professor and Associate Chairman of the department.

Dr. Banta is a member of the American Society of Mechanical Engineers (ASME), the American Society of Safety Engineers (ASEE), and the Society for Glass Science and Practice. He is a Registered Professional Engineer in the state of West Virginia.